

Model Zoo User Guide

Release v2.16.0
16 July 2025

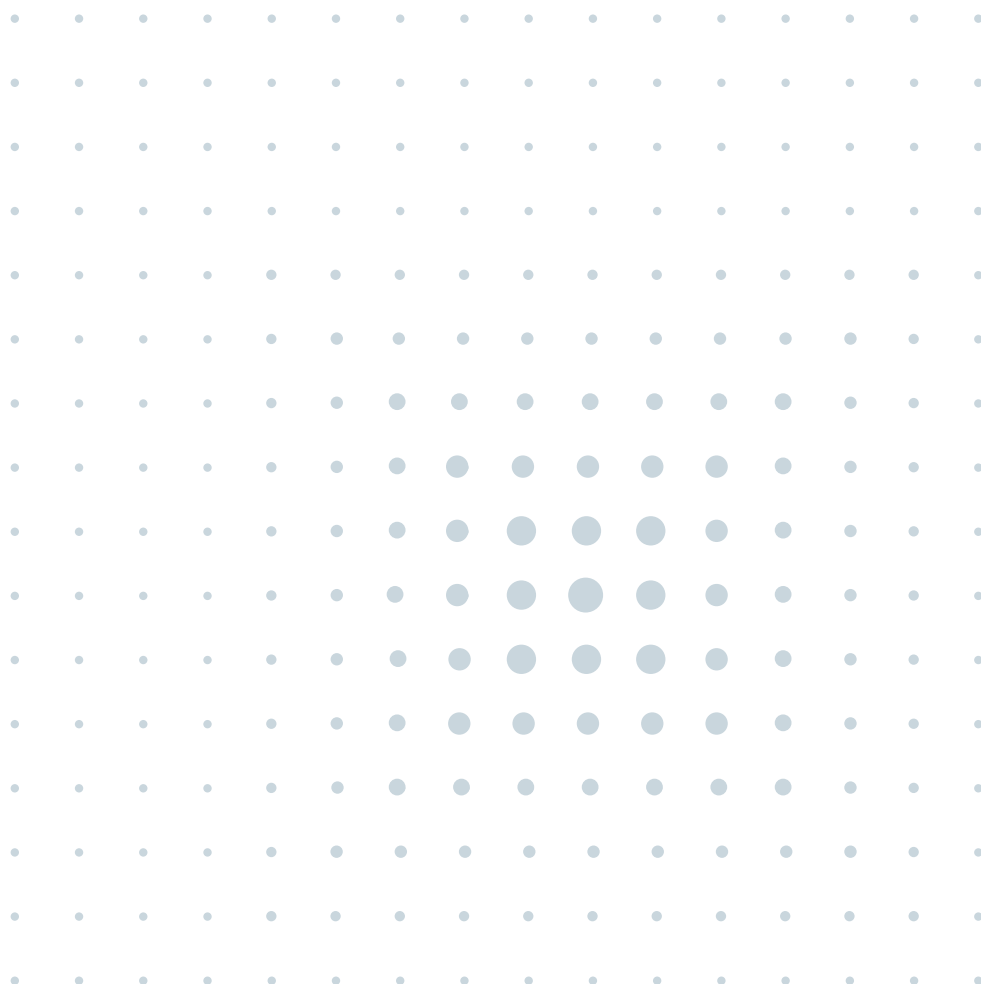


Table of Contents

1 Hailo Model Zoo	2
1.1 Retraining	2
1.2 Benchmarks	3
1.3 Quick Start Guide	3
1.4 License	3
1.5 Support & Issues	4
1.6 Changelog	4
2 Getting Started	14
2.1 System Requirements	14
2.2 Install Instructions	14
3 Usage	17
3.1 Flow Diagram	17
3.2 Parsing	17
3.3 Optimization	18
3.4 Profiling	18
3.5 Compilation	19
3.6 Evaluation	19
3.7 Visualization	20
3.8 Info	20
3.9 Compile multiple networks together	21
3.10 TFRecord to NPY conversion	21
4 Model Optimization	22
4.1 Introduction	22
4.2 Optimization Workflow	22
4.3 Citations	24
5 Hailo Models	25
5.1 License Plate Detection	26
5.2 Licesen Plate Recognition	29
5.3 Person-Face Detection	33
5.4 Person-ReID	37
5.5 Vehicle Detection	41
6 Datasets	45
6.1 ImageNet	46
6.2 COCO2017	46
6.3 Cityscapes	47
6.4 WIDERFACE	48
6.5 VisDrone	49
6.6 Pascal VOC augmented dataset	50
6.7 D2S augmented dataset	50
6.8 NYU Depth V2	51
6.9 AFLW2k3d and 300W-LP	51
6.10 Hand Landmark	53
6.11 Market1501	53
6.12 PETA	54
6.13 CelebA	54
6.14 LFW	55
6.15 BSD100	55

6.16	CLIP_CIFAR100	56
6.17	LOL	56
6.18	BSD68	57
6.19	CBSD68	57
6.20	KITTI_STEREO	58
6.21	KINETICS400	59
6.22	NUSCENES	60
7	Benchmarks	61
7.1	Example	61
7.2	Using Datasets from the Hailo Model Zoo	61
8	Hailo Model Zoo YAML Description	62
8.1	Properties	62
8.2	YAML hierarchies	63
8.3	Notes for Retraining	64
9	Retrain on Custom Dataset	65
9.1	YOLOv3 Retraining	65
9.2	YOLOv4-leaky Retraining	68
9.3	YOLOv5 Retraining	70
9.4	YOLOv8 Retraining	73
9.5	YOLOX Retraining	75
9.6	DAMO-YOLO Retraining	77
9.7	Nanodet Retraining	80
9.8	FCN Retraining	82
9.9	YOLACT Retraining	85
9.10	YOLOv8-seg Retraining	87
9.11	Centerpose Retraining	90
9.12	MSPN Retraining	92
9.13	Arface Retraining	94

Disclaimer and Proprietary Information Notice

Copyright

© 2024 Hailo Technologies Ltd ("Hailo"). All Rights Reserved.

No part of this document may be reproduced or transmitted in any form without the expressed, written permission of Hailo. Nothing contained in this document should be construed as granting any license or right to use proprietary information for that matter, without the written permission of Hailo.

This version of the document supersedes all previous versions.

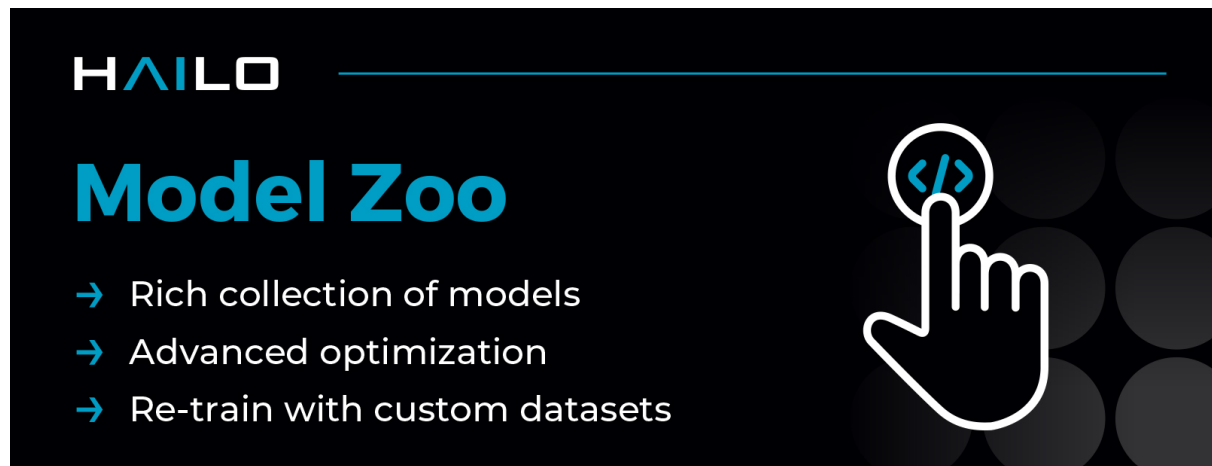
General Notice

Hailo, to the fullest extent permitted by law, provides this document "as-is" and disclaims all warranties, either express or implied, statutory or otherwise, including but not limited to the implied warranties of merchantability, non-infringement of third parties' rights, and fitness for particular purpose.

Although Hailo used reasonable efforts to ensure the accuracy of the content of this document, it is possible that this document may contain technical inaccuracies or other errors. Hailo assumes no liability for any error in this document, and for damages, whether direct, indirect, incidental, consequential or otherwise, that may result from such error, including, but not limited to loss of data or profits.

The content in this document is subject to change without prior notice and Hailo reserves the right to make changes to content of this document without providing a notification to its users.

1. Hailo Model Zoo



python 3.8 | 3.9 | 3.10

Tensorflow 2.12.0

CUDA 11.8

Hailo Dataflow Compiler 3.32.0

HailoRT (optional) 4.22.0

License MIT

The Hailo Model Zoo provides pre-trained models for high-performance deep learning applications. Using the Hailo Model Zoo you can measure the full precision accuracy of each model, the quantized accuracy using the Hailo Emulator and measure the accuracy on the Hailo-8 device. Finally, you will be able to generate the Hailo Executable Format (HEF) binary file to speed-up development and generate high quality applications accelerated with Hailo-8. The Hailo Model Zoo also provides re-training instructions to train the models on custom datasets and models that were trained for specific use-cases on internal datasets.

Models Hailo provides different pre-trained models in ONNX / TF formats and pre-compiled HEF (Hailo Executable Format) binary file to execute on the Hailo devices.

The models are divided to:

- Public models - which were trained on publicly available datasets.
 - For Hailo-8 - Classification, Object Detection, Segmentation, other tasks
 - For Hailo-8L - Classification, Object Detection, Segmentation, other tasks
 - For Hailo-15H - Classification, Object Detection, Segmentation, other tasks
 - For Hailo-15M - Classification, Object Detection, Segmentation, other tasks
 - For Hailo-10H - Classification, Object Detection, Segmentation, other tasks
 - [HAILO MODELS](#) which were trained in-house for specific use-cases on internal datasets.
- Each Hailo Model is accompanied with retraining instructions.

1.1. Retraining

Hailo also provides [RETRAINING INSTRUCTIONS](#) to train a network from the Hailo Model Zoo with custom dataset.

1.2. Benchmarks

List of Hailo's benchmarks can be found in hailo.ai.

In order to reproduce the measurements please refer to the following [page](#).

1.3. Quick Start Guide

- Install Hailo Dataflow Compiler and enter the virtualenv. In case you are not Hailo customer please contact hailo.ai
- Install HailoRT (optional). Required only if you want to run on Hailo-8. In case you are not Hailo customer please contact hailo.ai
- Clone the Hailo Model Zoo

```
git clone https://github.com/hailo-ai/hailo_model_zoo.git
```

- Run the setup script

```
cd hailo_model_zoo; pip install -e .
```

- Run the Hailo Model Zoo. For example, print the information of the MobileNet-v1 model:

```
hailomz info mobilenet_v1
```

1.3.1. Getting Started

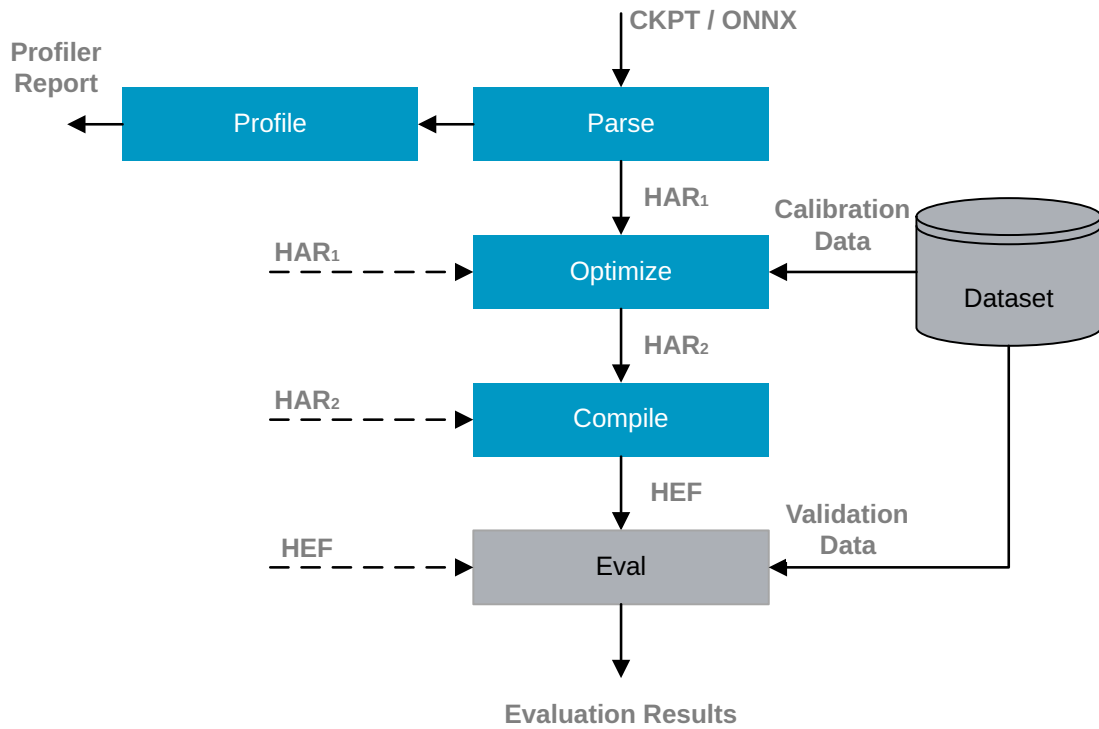
For full functionality please see the [INSTALLATION GUIDE](#) page (full install instructions and usage examples). The Hailo Model Zoo is using the Hailo Dataflow Compiler for parsing, model optimization, emulation and compilation of the deep learning models. Full functionality includes:

- Parse: model translation of the input model into Hailo's internal representation.
- Profiler: generate profiler report of the model. The report contains information about your model and expected performance on the Hailo hardware.
- Optimize: optimize the deep learning model for inference and generate a numeric translation of the input model into a compressed integer representation.
For further information please see our [OPTIMIZATION](#) page.
- Compile: run the Hailo compiler to generate the Hailo Executable Format file (HEF) which can be executed on the Hailo hardware.
- Evaluate: infer the model using the Hailo Emulator or the Hailo hardware and produce the model accuracy.

For further information about the Hailo Dataflow Compiler please contact hailo.ai.

1.4. License

The Hailo Model Zoo is released under the MIT license. Please see the [LICENSE](#) file for more information.



1.5. Support & Issues

❏ **Issue reporting is disabled in this repository.**

For bug reports, feature requests, or discussions, please visit our [Hailo Community Forum](#).

1.6. Changelog

For further information please see our [CHANGELOG](#) page. **v2.16**

- Update to use Dataflow Compiler v3.32.0 ([developer-zone](#))
- Update to use HailoRT 4.22.0 ([developer-zone](#))
- Removed support for hailo15h, hailo15m, hailo15l and hailo10h
- Removed support for clip_vit_l_14 and clip_vit_l_14_laion2B models
- Bug fixes

v2.15.1

- Included missing files for CLIP models

v2.15

- Update to use Dataflow Compiler v3.31.0 ([developer-zone](#))
- Update to use HailoRT 4.21.0 ([developer-zone](#))
- New Models:
 - [CLIP](#) - ViT-Base-16, ViT-Base-32, ViT-Large-14 (336x336 resolution) - Contrastive Language-Image Pre-training model [Hailo-15H and Hailo-10H only]

- [Real-ESRGAN](#) - x4 - Super Resolution model [Hailo-15H and Hailo-10H only]
- [PoolFormer](#) - s12 - Vision Transformer classification model [Hailo-15M/H and Hailo-10H only]
- Bug fixes

v2.14

- Update to use Dataflow Compiler v3.30.0 ([developer-zone](#))
- Update to use HailoRT 4.20.0 ([developer-zone](#))
- New cascade API (experimental)
 - Currently supports PETRv2, bird-eye-view network for 3D object detection, see `petrv2_repvvggB0.yaml` for configurations.
 - The user needs existing hars/hefs: both `petrv2_repvvggB0_backbone_pp_800x320` & `petrv2_repvvggB0_transformer_pp_800x320`
 - full_precision evaluation: `hailomz cascade eval petrv2`
 - hardware evaluation: `hailomz cascade eval petrv2 --override target=hardware`
- New task:
 - Human Action Recognition
 - ✦ Added support for (partial) Kinetics-400 dataset
 - ✦ Added r3d_18 to support this task
- New Models:
 - [YOLOv11](#) - nano, small, medium, large, x-large - Latest YOLO detectors
 - [CLIP](#) - ViT-Large-14-Laion2B - Contrastive Language-Image Pre-training model [H15H and H10H only]
 - [SWIN](#) - tiny, small - Shifted-Windows Transformer based classification model
 - [DaViT](#) - tiny - Dual Attention Vision Transformer classification model [H15H and H10H only]
 - [LeViT](#) - levit128, levit192, levit384 - Transformer based classification model
 - [EfficientFormer](#) - l1 - Transformer based classification model
 - [Real-ESRGAN](#) - x2 - Super Resolution model
 - [R3D_18](#) - r3d_18 - Video Classification network for Human Action Recognition [H8 only]
- Bug fixes

v2.13

- Update to use Dataflow Compiler v3.29.0 ([developer-zone](#))
- Update to use HailoRT 4.19.0 ([developer-zone](#))
- Using `jit_compile` which reduces dramatically the emulation inference time of the Hailo Model Zoo models.
- New tasks:
 - BEV: Multi-View 3D Object Detection
 - ✦ Added support for NuScenes dataset
 - ✦ Added PETRv2 with the following configuration:
 1. Backbone: RepVGG-B0 (800x320 input resolution)
 2. Transformer: 3 decoder layers, detection queries=304, replaced LN with UN
- New Models:
 - [CAS-ViT](#) - S, M, T - Convolutional-Attention based classification model
 - [YOLOv10](#) - base, x-large - Latest YOLO detectors

- CLIP Text Encoders - ResNet50x4, ViT-Large
- New retraining Docker containers for:
 - PETR - Multi-View 3D Object Detection
- Introduced new flags for hailomz CLI:
 - `--ap-per-class` for measuring average-precision per-class. Relevant for object detection and instance segmentation tasks.
- Bug fixes

v2.12

- Update to use Dataflow Compiler v3.28.0 ([developer-zone](#))
- Update to use HailoRT 4.18.0 ([developer-zone](#))
- Target hardware now supports Hailo-10H device
- New Models:
 - Original ViT models - tiny, small, base - Transformer based classification models
 - DeiT models - tiny, small, base - Transformer based classification models
 - DETR (resnet50) - Transformer based object detection model
 - fastvit_sa12 - Fast transformer based classification model
 - levit256 - Transformer based classification model
 - YOLOv10 - nano, small - Latest YOLO detectors
 - RepGhostNet1.0x, RepGhostNet2.0x - Hardware-Efficient classification models
- New postprocessing support on NN Core:
 - yolov6 tag 0.2.1
- Added support for person attribute visualization
- Bug fixes

v2.11

- Update to use Dataflow Compiler v3.27.0 ([developer-zone](#))
- Update to use HailoRT 4.18.0 ([developer-zone](#))
- New Models:
 - FastSAM-s - Zero-shot Instance Segmentation
 - Yolov9c - Latest Object Detection model of the YOLO family
- Using HailoRT-pp for postprocessing of the following variants:
 - nanodet

Postprocessing JSON configurations are now part of the `cfg` directory.

- Introduced new flags for hailomz CLI:
 - `--start-node-names` and `--end-node-names` for customizing parsing behavior.
 - `--classes` for adjusting the number of classes in post-processing configuration.

The `--performance` flag, previously utilized for compiling models with their enhanced model script if available, now offers an additional functionality. In instances where a model lacks an optimized model script, this flag triggers the compiler's Performance Mode to achieve the best performance

These flags simplify the process of compiling models generated from our retrain dockers.

- Bug fixes

v2.10

- Update to use Dataflow Compiler v3.26.0 ([developer-zone](#))
- Update to use HailoRT 4.16.0 ([developer-zone](#))
- Using HailoRT-pp for postprocessing of the following variants:
 - yolov8
- Profiler change:
 - Removal of `--mode` flag from `hailomz profile` command, which generates a report according to provided HAR state.
- CLI change:
 - `hailo8` target is deprecated in favor of `hardware`
- Support KITTI Stereo Dataset
- New Models:
 - `vit_pose_small` - encoder based transformer with layernorm for pose estimation
 - `segformer_b0_bn` - encoder based transformer with batchnorm for semantic segmentation
- Bug fixes

v2.9

- Update to use Dataflow Compiler v3.25.0 ([developer-zone](#))
- Update to use HailoRT 4.15.0 ([developer-zone](#))
- A new CLI-compatible API that allows users to incorporate format conversion and reshaping capabilities into the input:

```
hailomz compile yolov5s --resize 1080 1920 --input-conversion nv12_to_rgb
```

- New transformer models added:
 - `vit_pose_small_bn` - encoder based transformer with batchnorm for pose estimation
 - `clip_resnet_50x4` - Contrastive Language-Image Pre-Training for zero-shot classification
- New retraining dockers for vit variants using unified normalization.
- New Models:
 - `yolov8s_pose` / `yolov8m_pose` - pose estimation
 - `scdepthv3` - depth-estimation
 - `dncnn3` / `dncnn_color_blind` - image denoising
 - `zero_dce_pp` - low-light enhancement
 - `stereonet` - stereo depth estimation
- Using HailoRT-pp for postprocessing of the following models:
 - `efficientdet_lite0` / `efficientdet_lite1` / `efficientdet_lite2`

v2.8

- Update to use Dataflow Compiler v3.24.0 ([developer-zone](#))
- Update to use HailoRT 4.14.0 ([developer-zone](#))
- The Hailo Model Zoo now supports the following vision transformers models:
 - `vit_tiny` / `vit_small` / `vit_base` - encoder based transformer with batchnorm for classification
 - `detr_resnet_v1_18_bn` - encoder/decoder transformer for object detection

- clip_resnet_50 - Contrastive Language-Image Pre-Training for zero-shot classification
- yolov5s_c3tr - object detection model with a MHSA block
- Using HailoRT-pp for postprocessing of the following variants:
 - yolov5
 - yolox
 - ssd
 - efficientdet
 - yolov7
- New Models:
 - repvgg_a1 / repvgg_a2 - classification
 - yolov8_seg: yolov8n_seg / yolov8s_seg / yolov8m_seg - instance segmentation
 - yolov6n_0.2.1 - object detection
 - zero_dce - low-light enhancement
- New retraining dockers for:
 - yolov8
 - yolov8_seg
- Enable compilation for hailo15h device
- Enable evaluation of models with RGBX / NV12 input format
- Bug fixes

v2.7

- Update to use Dataflow Compiler v3.23.0 ([developer-zone](#))
- Updated to use HailoRT 4.13.0 ([developer-zone](#))
- Inference flow was moved to new high-level APIs
- New object detection variants:
 - yolov8: yolov8n / yolov8s / yolov8m / yolov8l / yolov8x
 - damoyolo: damoyolo_tinynasL20_T / damoyolo_tinynasL25_S / damoyolo_tinynasL35_M
- New transformers based models:
 - vit_base - classification model
 - yolov5s_c3tr - object detection model with a self-attention block
- Examples for using HailoRT-pp - support for seamless integration of models and their corresponding postprocessing
 - yolov5m_hpp
- Configuration YAMLS and model-scripts for networks with YUY2 input format
- DAMO-YOLO retraining docker
- Bug fixes

v2.6.1

- Bug fixes

v2.6

- Update to use Dataflow Compiler v3.22.0 ([developer-zone](#))

- Updated to use HailoRT 4.12.0 ([developer-zone](#))
- ViT ([Vision Transformer](#)) - new classification network with transformers-encoder based architecture
- New instance segmentation variants:
 - yolov5n_seg
 - yolov5s_seg
 - yolov5m_seg
 - yolov5l_seg
- New object detection variants for high resolution images:
 - yolov7e6
 - yolov5n6_6.1
 - yolov5s6_6.1
 - yolov5m6_6.1
- New flag `--performance` to reproduce highest performance for a subset of networks
- Hailo model-zoo log is now written into `sdk_virtualenv/etc/hailo/modelzoo/hailo_examples.log`
- Bug fixes

v2.5

- Update to use Dataflow Compiler v3.20.1 ([developer-zone](#))
- Model scripts use new bgr to rgb conversion
- New Yolact variants - with all COCO classes:
 - yolact_regnetx_800mf
 - yolact_regnetx_1.6gf
- Bug fixes

v2.4

- Updated to use Dataflow Compiler v3.20 ([developer-zone](#))
- Required FPS was moved from models YAML into the models scripts
- Model scripts use new change activation syntax
- New models:
 - Face Detection - scrfd_500m / scrfd_2.5g / scrfd_10g
- New tasks:
 1. Super-Resolution
 - Added support for BSD100 dataset
 - The following models were added: espcn_x2 / espcn_x3 / espcn_x4
 2. Face Recognition
 - Support for LFW dataset
 - The following models were added:
 1. arcface_r50
 2. arcface_mobilefacenet
 - Retraining docker for arcface architecture

- Added support for new hw-arch - hailo8l

v2.3

- Updated to use Dataflow Compiler v3.19 ([developer-zone](#))
- New models:
 - yolov6n
 - yolov7 / yolov7-tiny
 - nanodet_repvgg_a1_640
 - efficientdet_lite0 / efficientdet_lite1 / efficientdet_lite2
- New tasks:
 - mspn_regnetx_800mf - single person pose estimation
 - face_attr_resnet_v1_18 - face attribute recognition
- Single person pose estimation training docker (mspn_regnetx_800mf)
- Bug fixes

v2.2

- Updated to use Dataflow Compiler v3.18 ([developer-zone](#))
- CLI change:
 - Hailo model zoo CLI is now working with an entry point - hailomz
 - quantize sub command was changed to optimize
 - Hailo model zoo data directory by default will be ~/ .hailomz
- New models:
 - yolov5xs_wo_spp_nms - a model which contains bbox decoding and confidence thresholding on Hailo-8
 - osnet_x1_0 - person ReID network
 - yolov5m_6.1 - yolov5m network from the latest tag of the repo (6.1) including silu activation
- New tasks:
 - person_attr_resnet_v1_18 - person attribute recognition
- ReID training docker for the Hailo model repvgg_a0_person_reid_512/2048

NOTE: Ubuntu 18.04 will be deprecated in Hailo Model Zoo future version

NOTE: Python 3.6 will be deprecated in Hailo Model Zoo future version

v2.1

- Updated to use Dataflow Compiler v3.17 ([developer-zone](#))
- Parser commands were moved into model scripts
- Support Market-1501 Dataset
- Support a new model zoo task - ReID
- New models:
 - yolov5s_personface - person and face detector
 - repvgg_a0_person_reid_512 / repvgg_a0_person_reid_2048 - ReID networks which outputs a person embedding
These models were trained in-house as part of our upcoming new application
 - stdc1 - Segmentation architecture for Cityscapes

v2.0

- Updated to use Dataflow Compiler v3.16 ([developer-zone](#)) with TF version 2.5 which require CUDA11.2
- Updated to use HailoRT 4.6 ([developer-zone](#))
- Retraining Dockers - each retraining docker has a corresponding README file near it. New retraining dockers:
 - SSD
 - YOLOX
 - FCN
- New models:
 - yolov5l
- Introducing Hailo Models, in-house pretrained networks with compatible Dockerfile for retraining
 - yolov5m_vehicles (vehicle detection)
 - tiny_yolov4_license_plates (license plate detection)
 - lprnet (license plate recognition)
- Added new documentation to the [YAML structure](#)

v1.5

- Remove HailoRT installation dependency.
- Retraining Dockers
 - YOLOv3
 - NanoDet
 - CenterPose
 - Yolact
- New models:
 - unet_mobilenet_v2
- Support Oxford-IIIT Pet Dataset
- New multi-network example: detection_pose_estimation which combines the following networks:
 - yolov5m_wo_spp_60p
 - centerpose_repvvgg_a0
- Improvements:
 - nanodet_repvvgg mAP increased by 2%
- New Tasks:
 - hand_landmark_lite from MediaPipe
 - palm_detection_lite from MediaPipe

Both tasks are without evaluation module.

v1.4

- Update to use Dataflow Compiler v3.14.0 ([developer-zone](#))
- Update to use HailoRT 4.3.0 ([developer-zone](#))
- Introducing [Hailo Models](#) - in house pretrained networks with compatible Dockerfile for easy retraining:

- yolov5m_vehicles - vehicle detector based on yolov5m architecture
- tiny_yolov4_license_plates - license plate detector based on tiny_yolov4 architecture
- New Task: face landmarks detection
 - tddfa_mobilenet_v1
 - Support 300W-LP and AFLW2k3d datasets
- New features:
 - Support compilation of several networks together - a.k.a *multinets*
 - CLI for printing *network information*
- Retraining Guide:
 - New training guide for yolov4 with compatible Dockerfile
 - Modifications for yolov5 retraining

v1.3

- Update to use Dataflow Compiler v3.12.0 (*developer-zone*)
- New task: indoor depth estimation
 - fast_depth
 - Support NYU Depth V2 Dataset
- New models:
 - resmlp12 - new architecture support *paper*
 - yolox_l_leaky
- Improvements:
 - ssd_mobilenet_v1 - in-chip NMS optimization (de-fusing)
- Model Optimization API Changes
 - Model Optimization parameters can be updated using the networks' model script files (*.alls)
 - Deprecated: quantization params in YAMLS
- Training Guide: new training guide for yolov5 with compatible Dockerfile

v1.2

- New features:
 - YUV to RGB on core can be added through YAML configuration.
 - Resize on core can be added through YAML configuration.
- Support D2S Dataset
- New task: instance segmentation
 - yolact_mobilenet_v1 (coco)
 - yolact_regnetx_800mf_20classes (coco)
 - yolact_regnetx_600mf_31classes (d2s)
- New models:
 - nanodet_repvgg
 - centernet_resnet_v1_50_postprocess
 - yolov3 - *darkent* based
 - yolox_s_wide_leaky

- deeplab_v3_mobilenet_v2_dilation
- centerpose_repvgg_a0
- yolov5s, yolov5m - original models from [link](#)
- yolov5m_yuv - contains resize and color conversion on HW
- Improvements:
 - tiny_yolov4
 - yolov4
- IBC and Equalization API change
- Bug fixes

v1.1

- Support VisDrone Dataset
- New task: pose estimation
 - centerpose_regnetx_200mf_fpn
 - centerpose_regnetx_800mf
 - centerpose_regnetx_1.6gf_fpn
- New task: face detection
 - lightfaceslim
 - retinaface_mobilenet_v1
- New models:
 - hardnet39ds
 - hardnet68
 - yolox_tiny_leaky
 - yolox_s_leaky
 - deeplab_v3_mobilenet_v2
- Use your own network manual for YOLOv3, YOLOv4_leaky and YOLOv5.

v1.0

- Initial release
- Support for object detection, semantic segmentation and classification networks

2. Getting Started

This document provides install instructions and basic usage examples of the Hailo Model Zoo.

2.1. System Requirements

- Ubuntu 20.04/22.04, 64 bit (supported also on Windows, under WSL2)
- Python 3.8/3.9/3.10, including `pip` and `virtualenv`
- Hailo Dataflow Compiler v3.30.0 (Obtain from hailo.ai)
- HailoRT 4.20.0 (Obtain from hailo.ai) - required only for inference on Hailo-8.
- The Hailo Model Zoo supports Hailo-8 / Hailo-10H connected via PCIe only.
- Nvidia's Pascal/Turing/Ampere GPU architecture (such as Titan X Pascal, GTX 1080 Ti, RTX 2080 Ti, or RTX A4000)
- GPU driver version 525
- CUDA 11.8
- CUDNN 8.9

2.2. Install Instructions

2.2.1. Hailo Software Suite

The model requires the corresponding Dataflow Compiler version, and the optional HailoRT version. Therefore it is recommended to use the [Hailo Software Suite](#), that includes all of Hailo's SW components and insures compatibility across products versions.

The Hailo Software Suite is composed of the Dataflow Compiler, HailoRT, TAPPAS and the Model Zoo (*see [diagram below](#)*).

2.2.2. Manual Installation

1. Install the Hailo Dataflow compiler and enter the virtualenv (visit hailo.ai for further instructions).
2. Install the HailoRT - required only for inference on Hailo-8 / Hailo-10H (visit hailo.ai for further instructions).
3. Clone the Hailo Model Zoo repo:

```
git clone https://github.com/hailo-ai/hailo_model_zoo.git
```

4. Run the setup script:

```
cd hailo_model_zoo; pip install -e .
```

5. For setting up datasets please see [DATA](#).
6. Verify Hailo-8 / Hailo-10 is connected via PCIe (required only to run on Hailo-8 / Hailo-10. Full-precision / emulation run on GPU.)

```
hailortcli fw-control identify
```

Note: `hailortcli` is HailoRT command-line tool for interacting with Hailo devices.

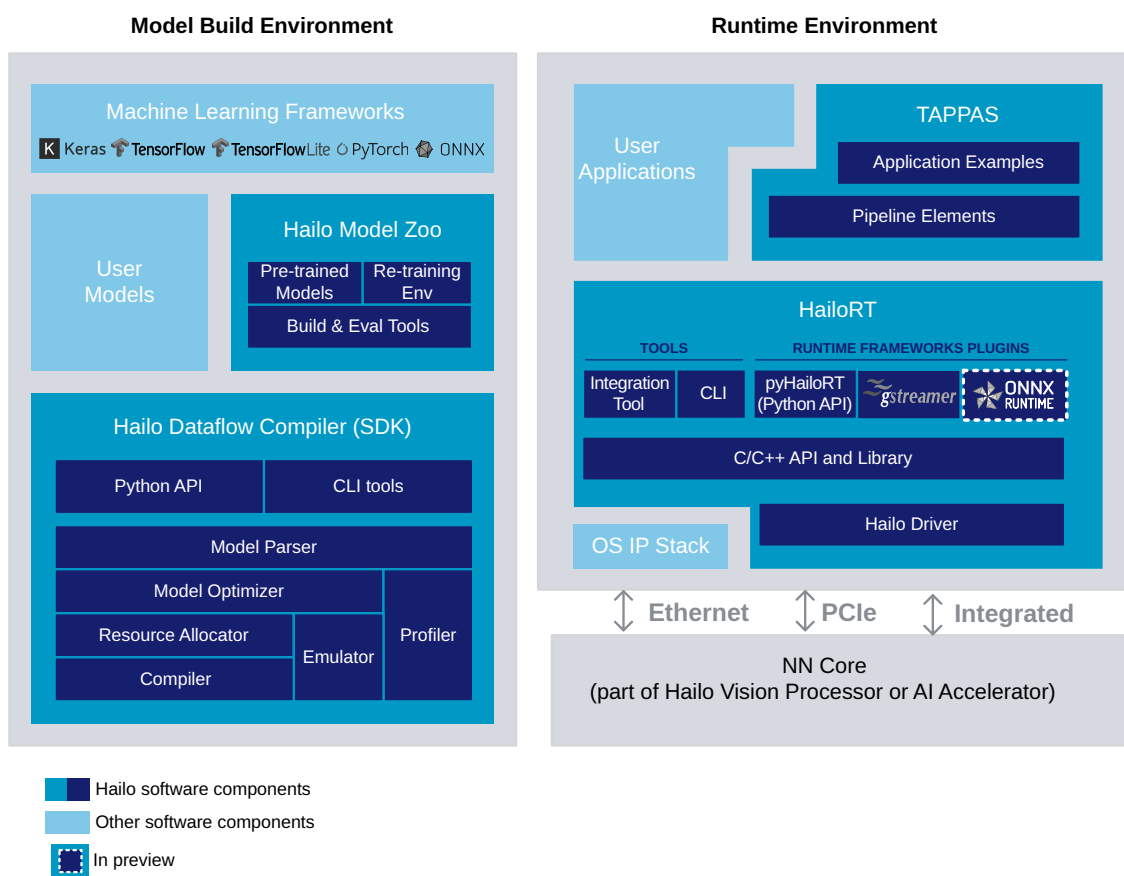


Fig. 1. Detailed block diagram of Hailo software packages

Expected output:

```
(hailo) Running command 'fw-control' with 'hailortcli'
Identifying board
Control Protocol Version: 2
Firmware Version: 4.6.0 (release,app)
Logger Version: 0
Board Name: Hailo-8
Device Architecture: HAILO8_B0
Serial Number: HLUTM20204900071
Part Number: HM218B1C2FA
Product Name: HAILO-8 AI ACCELERATOR M.2 MODULE
```

2.2.3. Upgrade Instructions

If you want to upgrade to a specific Hailo Model Zoo version within a suite or on top of a previous installation not in the suite.

1. Pull the specific repo branch:

```
git clone -b v2.6 https://github.com/hailo-ai/hailo_model_zoo.git
```

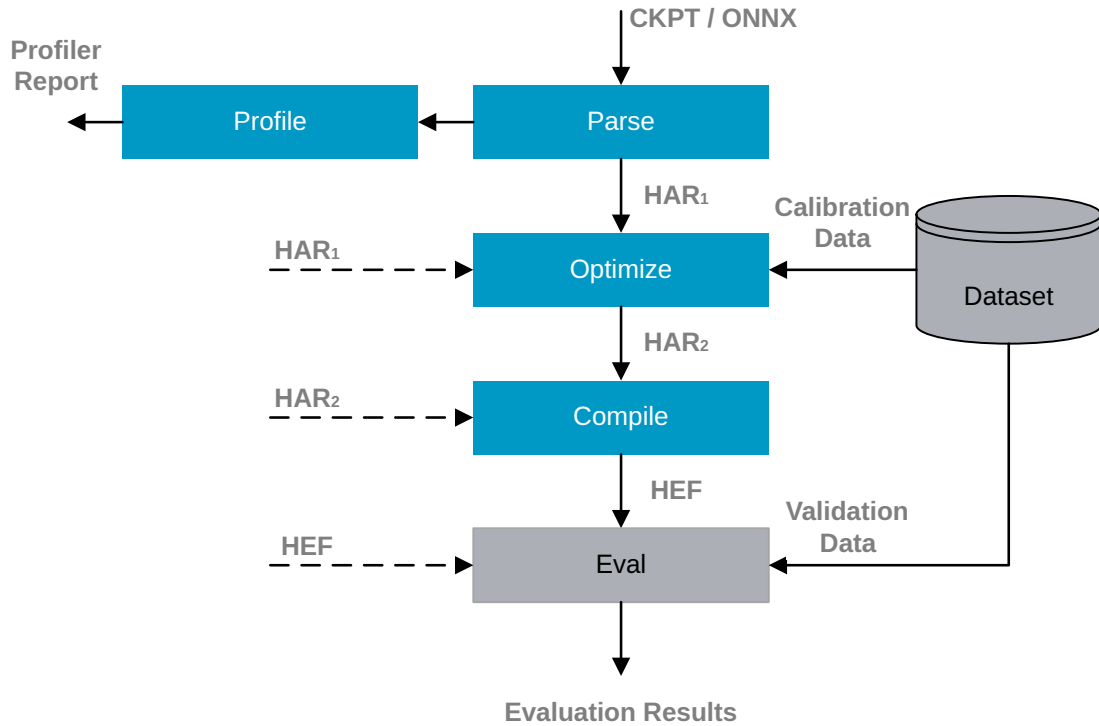
2. Run the setup script:

```
cd hailo_model_zoo; pip install -e .
```

3. Usage

3.1. Flow Diagram

The following scheme shows high-level view of the model-zoo evaluation process, and the different stages in between.



By default, each stage executes all of its previously necessary stages according to the above diagram. The post-parsing stages also have an option to start from the product of previous stages (i.e., the Hailo Archive (HAR) file), as explained below. The operations are configured through a YAML file that exist for each model in the `cfg` folder. For a description of the YAML structure please see [YAML](#).

NOTE: Hailo Model Zoo provides the following functionality for Model Zoo models only. If you wish to use your custom model, use the Dataflow Compiler directly.

3.2. Parsing

The pre-trained models are stored on AWS S3 and will be downloaded automatically when running the model zoo into your data directory. To parse models into Hailo's internal representation and generate the Hailo Archive (HAR) file:

```
hailomz parse <model_name>
```

- The default compilation target is Hailo-8. To compile for different architecture (Hailo-8L for example), use `--hw_arch hailo8l` as CLI argument:

```
hailomz parse <model_name> --hw-arch hailo8l
```

- To customize the parsing behavior, use `--start-node-names` and/or `--end-node-names` flags:

```
hailomz parse <model_name> --start-node-names <name1> --end-node-names <name1>  
↔ <name2>
```

3.3. Optimization

To optimize models, convert them from full precision into integer representation and generate a quantized Hailo Archive (HAR) file:

```
hailomz optimize <model_name>
```

To optimize the model starting from a previously generated HAR file:

```
hailomz optimize <model_name> --har /path/to/model.har
```

You can use your own images by giving a directory path to the optimization process, with the following supported formats (.jpg,.jpeg,.png):

```
hailomz optimize <model_name> --calib-path /path/to/calibration/imgs/dir/
```

- This step requires data for calibration. For additional information please see [OPTIMIZATION](#).

In order to achieve highest performance, use the performance flag:

```
hailomz optimize <model_name> --performance
```

The flag will be ignored on models that do not support this feature. The default and performance model scripts are located on *hailo_model_zoo/cfg/all/*

To add input conversion to the model, use the input conversion flag:

```
hailomz optimize <model_name> --input-conversion nv12_to_rgb
```

- Do not use the flag if an input conversion already exist in the alls or in the YAML.

To add input resize to the model, use the resize flag:

```
hailomz optimize <model_name> --resize 1080 1920
```

- Do not use the flag if resize already exist in the alls or in the YAML.

To adjust the number of classes in post-processing configuration, use classes flag:

```
hailomz optimize <model_name> --classes 80
```

- Use this flag only if post-process exists in the alls or in the YAML.

3.4. Profiling

To generate the model profiler report:

```
hailomz parse <model_name>  
hailo profiler path/to/model.har
```

- When profiling a Quantized HAR file (the result of the optimization process), the report contains information about your model and accuracy.
- When profiling a Compiled HAR file (the result of the compilation process), the report contains the expected performance on the Hailo hardware (as well as the accuracy information).

3.5. Compilation

To run the Hailo compiler and generate the Hailo Executable Format (HEF) file:

```
hailomz compile <model_name>
```

By default the compilation target is Hailo-8. To compile for a different architecture use `--hw-arch` command line argument:

```
hailomz compile <model_name> --hw-arch hailo8l
```

To generate the HEF starting from a previously generated HAR file:

```
hailomz compile <model_name> --har /path/to/model.har
```

- When working with a generated HAR, the previously chosen architecture will be used.

In order to achieve the best performance, use the performance flag:

```
hailomz optimize <model_name> --performance --hw-arch hardware
```

The flag will be ignored on models that do not support this feature. The default and performance model scripts are located on *hailo_model_zoo/cfg/all/*

To add input conversion to the model, use the input conversion flag:

```
hailomz compile <model_name> --input-conversion nv12_to_rgb
```

Do not use the flag if an input conversion already exist in the alls or in the YAML.

To add input resize to the model, use the resize flag:

```
hailomz compile <model_name> --resize 1080 1920
```

Do not use the flag if resize already exist in the alls or in the YAML.

3.6. Evaluation

To evaluate models in full precision:

```
hailomz eval <model_name>
```

To evaluate models starting from a previously generated Hailo Archive (HAR) file:

```
hailomz eval <model_name> --har /path/to/model.har
```

To evaluate models with the Hailo emulator (after quantization to integer representation - fast_numeric):

```
hailomz eval <model_name> --target emulator
```

To evaluate models on Hailo-8 / Hailo-10:

```
hailomz eval <model_name> --target hardware
```

If multiple devices are available, it's possible to select a specific one. Make sure to run on a device compatible to the compiled model.

```
# Device id looks something like 0000:41:00.0
hailomz eval <model_name> --target <device_id>
# This command can be used to list available devices
hailomz eval --help
```

To limit the number of images for evaluation use the following flag:

```
hailomz eval <model_name> --data-count <num-images>
```

To eval model with additional input conversion, use the input conversion flag:

```
hailomz eval <model_name> --input-conversion nv12_to_rgb
```

Do not use the flag if an input conversion already exist in the alls or in the YAML.

To eval model with input resize, use the resize flag:

```
hailomz eval <model_name> --resize 1080 1920
```

Do not use the flag if resize already exist in the alls or in the YAML.

To evaluate a cascade model please refer to CASCADE.

To explore other options (for example: changing the default batch-size) use:

```
hailomz eval --help
```

- Currently MZ evaluation can be done only on Hailo-8 and Hailo-10H.

3.7. Visualization

To run visualization (without evaluation) and generate the output images:

```
hailomz eval <model_name> --visualize
```

To create a video file from the network predictions:

```
hailomz eval <model_name> --visualize --video-outpath /path/to/video_output.mp4
```

3.8. Info

You can easily print information of any network exists in the model zoo, to get a sense of its input/output shape, parameters, operations, framework etc.

To print a model-zoo network information:

```
hailomz info <model_name>
```

Here is an example for printing information about mobilenet_v1:

```
hailomz info mobilenet_v1
```

Expected output:

```
<Hailo Model Zoo Info> Printing mobilenet_v1 Information
<Hailo Model Zoo Info>
  task:          classification
  input_shape:   224x224x3
  output_shape:  1x1x1001
  operations:    0.57G
  parameters:    4.22M
  framework:     tensorflow
  training_data: imagenet train
  validation_data: imagenet val
```

(continues on next page)

(continued from previous page)

```
eval_metric:      Accuracy (top1)
full_precision_result: 71.02
source:          https://github.com/tensorflow/models/tree/v1.13.0/research/
↳ slim
license_url:      https://github.com/tensorflow/models/blob/v1.13.0/LICENSE
```

3.9. Compile multiple networks together

We can use multiple disjoint models in the same binary. This is useful for running several small models on the device.

```
python hailo_model_zoo/multi_main.py <config_name>
```

3.10. TFRecord to NPY conversion

In some situations you might want to convert the tfrecord file to npy file (for example, when explicitly using the Dataflow Compiler for quantization). In order to do so, run the command:

```
python hailo_model_zoo/tools/conversion_tool.py /path/to/tfrecord_file resnet_v1_
↳ 50 --npy
```


4. Model Optimization

4.1. Introduction

Model optimization is the stage of converting a full precision model to an optimized model which will be compiled to the Hailo device.

This stage includes numeric translation of the input model into a compressed integer representation as well as optimizing the model architecture to best fit the Hailo hardware.

Compressing deep learning model induce degradation for the model's accuracy.

For example, in the following table we compare the accuracy of full precision ResNet V1 18 with the compressed 8-bits weights and activation representation on ImageNet-1K:

Precision	Top-1
Full precision	68.85
8-bit (emulator)	68.54

The main goal of the model optimization step is to prepare the model for compilation with minimum degradation as possible.

4.2. Optimization Workflow

The model optimization has two main steps: full precision optimization and quantization optimization.

- Full precision optimization includes any changes to the model in the floating point precision domain, for example, Equalization ([Meller2019](#)), TSE ([Vosco2021](#)) and pruning.
- Quantization includes compressing the model from floating point to integer representation of the weights and activations (4/8/16 bits) and algorithms to improve the model's accuracy, such as IBC ([Finkelstein2019](#)) and QFT ([Finkelstein2022](#)).

Both steps may degrade the model accuracy, therefore, evaluation is needed to verify the model accuracy. This workflow is depicted in the following diagram:

NOTE: Hailo Model Zoo provides the following functionality for Model Zoo models only. If you wish to use your custom model, use the Dataflow Compiler directly.

1. First step includes full precision validation. This step is important to make sure parsing was successful and we built the pre/post processing and evaluation of the model correctly. In the Hailo Model Zoo, we can execute the following which will infer a specific model in full precision to verify that the accuracy is correct (this will be our baseline for measuring degradation):

```
hailomz eval <model_name>
```

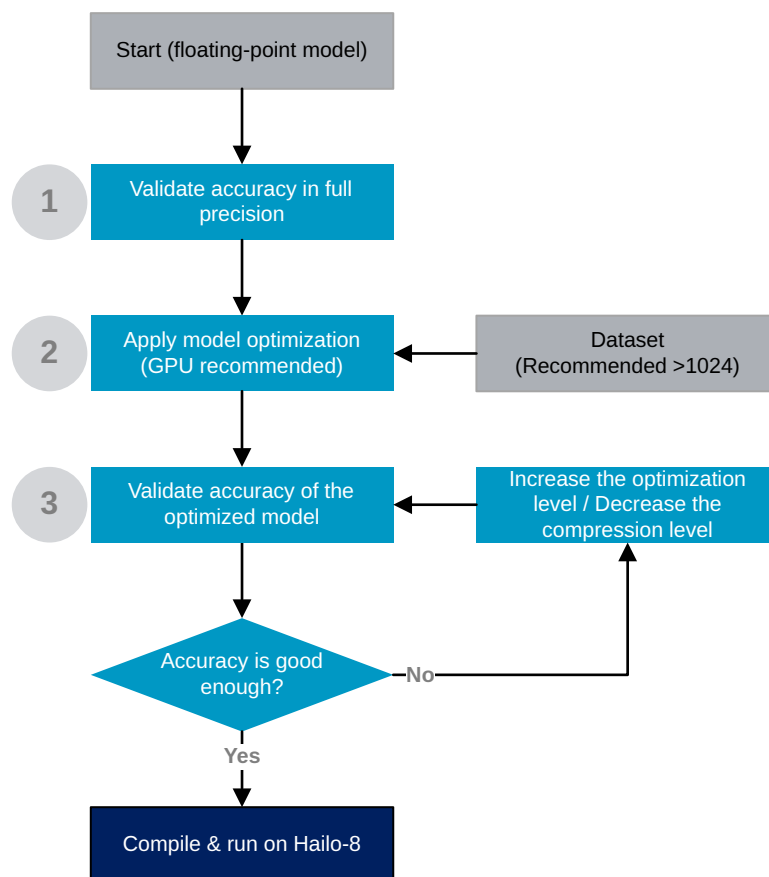
2. Next, we call the model optimization API to generate an optimized model. Note, it is recommended to run this step on a GPU machine with dataset size of at least 1024 images.

```
hailomz optimize <model_name>
```

3. Lastly, we verify the accuracy of the optimized model. In case the results are not good enough we should repeat the process with different configurations of the optimization/compression levels:

```
hailomz eval <model_name> --target emulator --har <model_name>.har
```

Once optimization is finished and met our accuracy requirements, we can compile the optimized model. For example:



```
hailomz compile <model_name> --har <model_name>.har
```

4.3. Citations

4.3.1. Vosco2021

```
@InProceedings{Vosco2021,
  title = {Tiled Squeeze-and-Excite: Channel Attention With Local Spatial Context},
  author = {Niv Vosco and Alon Shenkler and Mark Grobman},
  booktitle = {ICCV},
  year = {2021}
}
```

4.3.2. Finkelstein2019

```
@InProceedings{Finkelstein2019,
  title = {Fighting Quantization Bias With Bias},
  author = {Alexander Finkelstein and Uri Almog and Mark Grobman},
  booktitle = {CVPR},
  year = {2019}
}
```

4.3.3. Meller2019

```
@InProceedings{Meller2019,
  title = {Same, Same But Different - Recovering Neural Network Quantization Error
  ↪ Through Weight Factorization},
  author = {Eldad Meller and Alexander Finkelstein and Uri Almog and Mark Grobman},
  booktitle = {ICML},
  year = {2019}
}
```

5. Hailo Models

Here, we give the full list of models trained in-house for specific use-cases. Each model is accompanied with its own README, retraining docker and retraining guide.

- FLOPs in the table are counted as MAC operations.

- [Person & Face Detection](#)
- [License Plate Recognition](#)
- [Person Re-Identification](#)

Important: Retraining is not available inside the docker version of Hailo Software Suite. In case you use it, clone the hailo_model_zoo outside of the docker, and perform the retraining there: `git clone https://github.com/hailo-ai/hailo_model_zoo.git`

1. Object Detection

Network Name	mAP*	Input Resolution (HxWxC)	Params (M)	FLOPs (G)
yolov5m_vehicles	46.5	640x640x3	21.47	25.63
tiny_yolov4_license_plates	73.45	416x416x3	5.87	3.4
yolov5s_personface	47.5	640x640x3	7.25	8.38

2. License Plate Recognition

Network Name	Accuracy*	Input Resolution (HxWxC)	Params (M)	FLOPs (G)
lprnet	99.96	75x300x3	7.14	18.29

* Evaluated on internal dataset

3. Person Re-ID

Network Name	Accuracy*	Input Resolution (HxWxC)	Params (M)	FLOPs (G)
repvgg_a0_person_reid_512	89.9	256x128x3	7.68	0.89
repvgg_a0_person_reid_2048	90.02	256x128x3	9.65	0.89

* Evaluated on Market-1501

5.1. License Plate Detection



Hailo's license detection network (*tiny_yolov4_license_plates*) is based on Tiny-YOLOv4 and was trained in-house using Darknet with a single class. It expects a single vehicle and can work under various weather and lighting conditions, on different vehicle types and numerous camera angles.

5.1.1. Model Details

Architecture

- Tiny-YOLOv4
 - Number of parameters: 5.87M
 - GMACS: 3.4
 - Accuracy* : 73.45 mAP
- * Evaluated on internal dataset containing 5000 images

Inputs

- RGB image with size of 416x416x3
- Image normalization occurs on-chip

Outputs

- Two output tensors with sizes of 13x13x18 and 26x26x18.
 - Each output contains 3 anchors that hold the following information:
 - Bounding box coordinates ((x,y) centers, height, width)
 - Box objectness confidence score
 - Class probability confidence score
 - The above 6 values per anchor are concatenated into the 18 output channels
-

Download

The pre-compiled network can be downloaded from [here](#).

Use the following command to measure model performance on hailo's HW:

```
hailortcli benchmark tiny_yolov4_license_plates.hef
```

5.1.2. Train License Plate Detection on a Custom Dataset

Here we describe how to finetune Hailo's license plate detection network on your own custom dataset.

Prerequisites

- docker ([installation instructions](#))
- nvidia-docker2 ([installation instructions](#))

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

Environment Preparations

1. Build the docker image

```
cd hailo_model_zoo/hailo_models/license_plate_detection/
docker build --build-arg timezone='cat /etc/timezone' -t license_plate_
→detection:v0 .
```

- This command will build the docker image with the necessary requirements using the Dockerfile exists in this directory.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all --ipc=host -v /path/to/local/
→data/dir:/path/to/docker/data/dir license_plate_detection:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `license_plate_detection:v0` the name of the docker image.

Finetuning and exporting to ONNX

1. Train the network on your dataset

Once the docker is started, you can train the license plate detector on your custom dataset. We recommend following the instructions for YOLOv4 training that can be found in [here](#). The important steps are specified below:

- Update `data/obj.data` with paths to your training and validation `.txt` files, which contain the list of the image paths*.

```
classes = 1
train = data/train.txt
valid = data/val.txt
names = data/obj.names
backup = backup/
```

* Tip: specify the paths to the training and validation images in the training and validation `.txt` files relative to `/workspace/darknet/`

- Place your training and validation images and labels in your data folder.
- Start training on your dataset starting from our pre-trained weights in `tiny_yolov4_license_plates.weights` (or download it from [here](#))

```
./darknet detector train data/obj.data ./cfg/tiny_yolov4_license_plates.cfg
→tiny_yolov4_license_plates.weights -map -clear
```

NOTE: If during training you get an error similar to

```
cuDNN status Error in: file: ./src/convolutional_kernels.cu : ( ) : line: 543 :
→build time: Jun 21 2022 - 20:09:28

cuDNN Error: CUDNN_STATUS_BAD_PARAM
Darknet error location: ./src/dark_cuda.c, cudnn_check_error, line #204
cuDNN Error: CUDNN_STATUS_BAD_PARAM: Success
```

- then please try changing `subdivisions` in the `.cfg` file (e.g., from 16 to 32).
- For further information, please see discussion [here](#).

2. Export to ONNX

Export the model to ONNX using the following command:

```
python ./pytorch-YOLOv4/demo_darknet2onnx.py cfg/tiny_yolov4_license_plates.
→cfg /path/to/trained.weights /path/to/some/image.jpg 1
```

Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model. In order to do so you need a working model-zoo environment. Choose the model YAML from our networks configuration directory, i.e. `hailo_model_zoo/cfg/networks/tiny_yolov4_license_plates.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt tiny_yolov4_license_plates_1_416_416.onnx --calib-path /
→path/to/calibration/imgs/dir/ --yaml path/to/tiny_yolov4_license_plates.yaml --
→start-node-names name1 name2 --end-node-names name1
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format

- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note:

- Since it's an Hailo model, calibration set must be manually supplied.
- On `tiny_yolov4_license_plates.yaml`, change `postprocessing` section if anchors changed, `evaluation.classes` if classes amount is changed, and `evaluation.labels_offset` if it was changed on retraining.
- On `yolo.yaml`, change `preprocessing.input_shape` if the network is trained on other resolution.

More details about YAML files are presented [here](#).

5.2. Licesen Plate Recognition

Plate Number: 568612



Hailo's license plate recognition network (*lprnet*) was trained in-house on a synthetic auto-generated dataset to predict registration numbers of license plates under various weather and lighting conditions.

5.2.1. Model Details

Architecture

- A convolutional network based on *LPRNet*, with several modifications:
 - A ResNet like backbone with 4 stages, each containing 2 residual blocks
 - Several kernel shape changes
 - Maximal license plate length of 19 digits
 - More details can be found [here](#)
- Number of parameters: 7.14M
- GMACS: 18.29
- Accuracy*: 99.96%
 - * Evaluated on internal dataset containing 1178 images

Inputs

- RGB license plate image with size of 75x300x3
- Image normalization occurs on-chip

Outputs

- A tensor with size 5x19x11
 - Post-processing outputs a tensor with size of 1x19x11
 - The 11 channels contain logits scores for 11 classes (10 digits + *blank* class)
 - A Connectionist temporal classification (CTC) greedy decoding outputs the final license plate number prediction
-

Download

The pre-compiled network can be downloaded from [here](#).

Use the following command to measure model performance on hailo's HW:

```
hailortcli benchmark lprnet.hef
```

Training on Custom Dataset

- Hailo's LPRNet was trained on a synthetic auto-generated dataset containing 4 million license plate images. Auto-generation of synthetic data for training is cheap, allows one to obtain a large annotated dataset easily and can be adapted quickly for other domains
- A notebook for auto-generation of synthetic training data for LPRNet can be found [here](#)
- For more details on the training data autogeneration, please see the training guide

5.2.2. Train License Plate Recognition on a Custom Dataset

Here we describe how to finetune Hailo's optical character reader (OCR) model for license plate recognition with your own custom dataset.

Prerequisites

- docker ([installation instructions](#))
- nvidia-docker2 ([installation instructions](#))

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

Environment Preparations

1. Build the docker image

```
cd hailo_model_zoo/hailo_models/license_plate_recognition/
docker build --build-arg timezone='cat /etc/timezone' -t license_plate_
➔ recognition:v0 .
```

- This command will build the docker image with the necessary requirements using the Dockerfile that exists in this directory.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all --ipc=host -v /path/to/local/
➔ data/dir:/path/to/docker/data/dir license_plate_recognition:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `license_plate_recognition:v0` the name of the docker image.

Finetuning and exporting to ONNX

1. Train the network on your dataset

Once the docker is started, you can train the OCR on your custom dataset.

- Create a folder with license plates images for training and testing. The folder should contain images whose file names correspond to the plate number, e.g. 12345678 .png.

NOTE: Please make sure the file names **do not** contain characters which are not numbers or letters.

- Alternatively, you can use the provided jupyter notebook in `dataset/lp_autogenerate.ipynb` as an example of how to auto-generate a synthetic dataset of license plates for training. The auto-generation uses several parameters to control the randomness in the dataset, such as allowed characters, font size, font color, character separation etc. (Please refer to the notebook for more details). In order to auto-generate the synthetic dataset, please provide the following:

- Clean license plate images with no characters in the `dataset/plates/` folder
- .ttf font files in the `dataset/fonts/` folder

NOTE: We recommend the autogenerated training set to contain at least 4 million images

- Start training on your dataset:
 - Start from our pre-trained weights in `pre_trained/lprnet.pth` (you can also download it from [here](#))

```
python train_LPRNet.py --train_img_dirs path/to/train/images --test_
➔ img_dirs path/to/test/images --max_epoch 30 --train_batch_size 64 --
➔ test_batch_size 32 --resume_epoch 15 --pretrained_model pre_trained/
➔ lprnet.pth --save_folder runs/exp0/ --test_interval 2000
```

- Or train from scratch

```
python train_LPRNet.py --train_img_dirs path/to/train/images --test_
↪img_dirs path/to/test/images --max_epoch 15 --save_folder runs/exp0/
```

2. Export to ONNX

Export the model to ONNX using the following command:

```
python export.py --onnx lprnet.onnx --weights /path/to/trained/model.pth
```

Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model. In order to do so you need a working model-zoo environment. Choose the model YAML from our networks configuration directory, i.e. `hailo_model_zoo/cfg/networks/lprnet.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt lprnet.onnx --calib-path /path/to/calibration/imgs/dir/ --
↪yaml path/to/lprnet.yaml --start-node-names name1 name2 --end-node-names name1
```

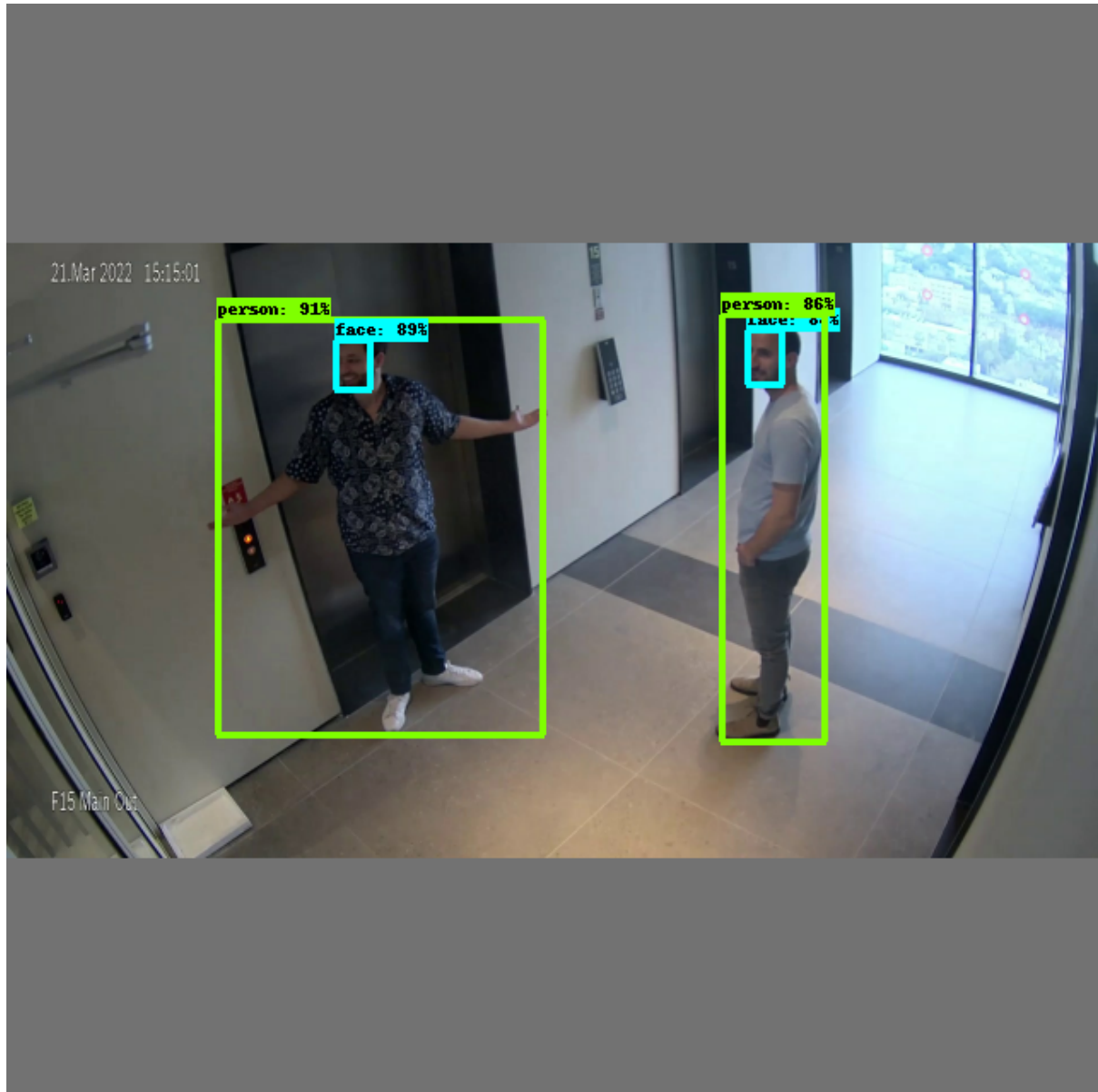
- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note:

- Since it's an Hailo model, calibration set must be manually supplied.

More details about YAML files are presented [here](#).

5.3. Person-Face Detection



Hailo's person-face detection network (*yolov5s_personface*) is based on YOLOv5s and was trained in-house with two classes [person, face]. It can work under various lighting conditions, number of people, and numerous camera angles.

5.3.1. Model Details

Architecture

- YOLOv5s
 - Number of parameters: 7.25M
 - GMACS: 8.38G
 - Accuracy*: 47.5 mAP
- * Evaluated on internal dataset containing 6000 images

Inputs

- RGB image with various input sizes
 - Image resize to 640x640x3 occurs on-chip
- Image normalization occurs on-chip

Outputs

- Three output tensors with sizes of 20x20x21, 40x40x21 and 80x80x21
 - Each output contains 3 anchors that hold the following information:
 - Bounding box coordinates ((x,y) centers, height, width)
 - Box objectness confidence score
 - Class probability confidence score per class
 - The above 7 values per anchor are concatenated into the 21 output channels
-

Comparison with Different Models

The table below shows the performance of our trained network on an internal validation set containing 6000 images, compared to other benchmark models from the model zoo*.

network	Person mAP (@IoU=0.5:0.95)
yolov5s_personface	34.2
yolov5s*	23.0
yolov5m*	25.6

* Benchmark models were trained on all COCO classes, and evaluated on our internal validation set, on 'Person' class only.

Download

The pre-compiled network can be downloaded from [here](#)

Use the following command to measure model performance on hailo's HW:

```
hailortcli benchmark yolov5s_personface.hef
```

5.3.2. Train Person-Face Detection on a Custom Dataset

Here we describe how to finetune Hailo's person-face detection network with your own custom dataset.

Prerequisites

- docker ([installation instructions](#))
- nvidia-docker2 ([installation instructions](#))

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/hailo_models/personface_detection/
docker build --build-arg timezone='cat /etc/timezone' -t personface_
→detection:v0 .
```

- This command will build the docker image with the necessary requirements using the Dockerfile that exists in this directory.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all --ipc=host -v /path/to/
→local/data/dir:/path/to/docker/data/dir personface_detection:v0
```

- docker run create a new docker container.
- --name <your_docker_name> name for your container.
- -it runs the command interactively.
- --gpus all allows access to all GPUs.
- --ipc=host sets the IPC mode for the container.
- -v /path/to/local/data/dir:/path/to/docker/data/dir maps /path/to/local/data/dir from the host to the container. You can use this command multiple times to mount multiple directories.
- personface_detection:v0 the name of the docker image.

Finetuning and exporting to ONNX

1. Train the network on your dataset

Once the docker is started, you can train the person-face detector on your custom dataset. We recommend following the instructions for YOLOV5 training that can be found in [here](#). The important steps are specified below:

- Update the dataset config file data/personface_data.yaml with the paths to your training and validation images files.

```
#update your data paths
train: /path/to/personface/images/train/
val: /path/to/personface/images/val
# number of classes
nc: 2
# class names
names: ['person', 'face']
```

- Start training on your dataset starting from our pre-trained weights in `weights/yolov5s_personface.pt` (you can also download it from [here](#))

```
python train.py --data ./data/personface_data.yaml --cfg ./models/yolov5s_
→personface.yaml --weights ./weights/yolov5s_personface.pt --epochs 300 --
→batch 128 --device 1,2,3,4
```

2. **Export to ONNX** Export the model to ONNX using the following command:

```
python models/export.py --weights ./runs/exp<#>/weights/best.pt --img-size 640
→--batch-size 1
```

- The best model's weights will be saved under the following path:
`./runs/exp<#>/weights/best.pt`
, where `<#>` is the experiment number.
- Export at 640x640 with batch size 1

Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model. In order to do so you need a working model-zoo environment.

Choose the model YAML from our networks configuration directory, i.e.

`hailo_model_zoo/cfg/networks/yolov5s_personface.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt yolov5s_personface.onnx --calib-path /path/to/calibration/
→imgs/dir/ --yaml path/to/yolov5s_personface.yaml --start-node-names name1 name2 --
→end-node-names name1 --classes 80
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- `--classes` - adjusting the number of classes in post-processing configuration (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note:

- Since it's an Hailo model, calibration set must be manually supplied.
- On `yolo.yaml`, change `preprocessing.input_shape` if changed on retraining

More details about YAML files are presented [here](#).

5.3.3. Anchors Extraction

The training flow will automatically try to find more fitting anchors values then the default anchors. In our TAPPAS environment we use the default anchors, but you should be aware that the resulted anchors might be different. The model anchors can be retrieved from the trained model using the following snippet:

```
m = torch.load("last.pt")["model"]
detect = list(m.children())[0][-1]
print(detect.anchor_grid)
```

5.4. Person-ReID



Hailo's person Re-Identification network is based on RepVGG_A0 and was trained in-house using several Person ReID datasets. It can work under various lighting conditions and numerous camera angles. 2 Models were trained independently - using 512 & 2048 embedding dimensions.

5.4.1. Model Details

Architecture (2048 / 512)

- RepVGG_A0
 - Number of parameters: 9.65M / 7.68M
 - GMACS: 0.89 / 0.89
 - Rank1* : 89.8% / 89.3%
- * Evaluated on Market1501 dataset

Inputs

- RGB image with various input sizes
 - Image resize to 256x128x3 occurs on-chip
- Image normalization occurs on-chip

Outputs

- Single embedding vector (2048 / 512 dim) per query

5.4.2. Comparison with Different Models

The table below shows the performance of our trained network on Market1501 dataset.

network	Person Rank1
repvgg_a0_person_reid_512	89.3
repvgg_a0_person_reid_2048	89.8

Download

The pre-compiled network can be download from:

- [512-dim](#)
- [2048-dim](#)

Use the following command to measure model performance on hailo's HW:

```
hailortcli benchmark repvgg_a0_person_reid_512.hef
hailortcli benchmark repvgg_a0_person_reid_2048.hef
```

5.4.3. Train Person-ReID on a Custom Dataset

Here we describe how to finetune Hailo's person-reid network with your own custom dataset.

Prerequisites

- [docker \(installation instructions\)](#)
- [nvidia-docker2 \(installation instructions\)](#)

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/hailo_models/reid/
docker build --build-arg timezone='cat /etc/timezone' -t person_reid:v0 .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.
- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.

This command will build the docker image with the necessary requirements using the Dockerfile that exists in this directory.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all --ipc=host -v /path/to/local/
→drive:<span
val="docker_vol_path">/path/to/docker/dir person_reid:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `personface_detection:v0` the name of the docker image.

Finetuning and exporting to ONNX

1. Train the network on your dataset

Once the docker is started, you can train the person-reid model on your custom dataset. We recommend following the instructions from the torchreid repo that can be found in [here](#).

- Insert your dataset as described in [use-your-own-dataset](#).
- Start training on your dataset starting from our pre-trained weights in `models/repvgg_a0_person_reid_512.pth` or `models/repvgg_a0_person_reid_2048.pth` (you can also download it from [512-dim](#) & [2048-dim](#)) - to do so, you can edit the added `yaml` configs/`repvgg_a0_hailo_pre_train.yaml` and take a look at the examples in [torchreid](#).

```
python scripts/main.py --config-file configs/repvgg_a0_hailo_pre_train.
→yaml
```

2. Export to ONNX

Export the model to ONNX using the following command:

```
python scripts/export.py --model_name <model_name> --weights /path/to/model.pth
```

Compile the Model using Hailo Model Zoo

In case you exported to onnx based on one of our provided RepVGG models, you can generate an HEF file for inference on Hailo-8 from your trained ONNX model. In order to do so you need a working model-zoo environment.

Choose the model YAML from our networks configuration directory, i.e.

hailo_model_zoo/cfg/networks/repvgg_a0_person_reid_512.yaml (or 2048), and run compilation using the model zoo:

```
hailomz compile --ckpt repvgg_a0_person_reid_512.onnx --calib-path /path/to/
↳calibration/imgs/dir/ --yaml path/to/repvgg_a0_person_reid_512.yaml --start-
↳node-names name1 name2 --end-node-names name1
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note:

- Since it's an Hailo model, calibration set must be manually supplied.
- On `market1501.yaml`, change `preprocessing.input_shape` if changed on retraining

More details about YAML files are presented [here](#).

5.5. Vehicle Detection



Hailo's vehicle detection network (*yolov5m_vehicles*) is based on YOLOv5m and was trained in-house with a single class. It can work under various weather and lighting conditions, and numerous camera angles.

5.5.1. Model Details

Architecture

- YOLOv5m
- Number of parameters: 21.47M
- GMACS: 25.63
- Accuracy* : 46.0 mAP
 - * Evaluated on internal dataset containing 5000 images

Inputs

- RGB image with size of 1080x1920x3
 - Image resize to 640x640x3 occurs on-chip
- Image normalization occurs on-chip

Outputs

- Three output tensors with sizes of 20x20x18, 40x40x18 and 80x80x18
 - Each output contains 3 anchors that hold the following information:
 - Bounding box coordinates ((x,y) centers, height, width)
 - Box objectness confidence score
 - Class probability confidence score
 - The above 6 values per anchor are concatenated into the 18 output channels
-

Comparison with Different Models

The table below shows the performance of our trained network on an internal validation set containing 5000 images, compared with the performance of other benchmark models from the model zoo*.

network	Vehicle mAP (@IoU=0.5:0.95)
yolov5m_vehicles	46.0
yolov5m	33.95
yolov4_leaky	33.13
yolov3_gluon	29.89

* Benchmark models were trained on all COCO classes

Download

The pre-compiled network can be downloaded from [here](#).

Use the following command to measure model performance on hailo's HW:

```
hailortcli benchmark yolov5m_vehicles.hef
```

5.5.2. Train Vehicle Detection on a Custom Dataset

Here we describe how to finetune Hailo's vehicle detection network with your own custom dataset.

Prerequisites

- docker ([installation instructions](#))
- nvidia-docker2 ([installation instructions](#))

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

Environment Preparations

1. Build the docker image

```
cd hailo_model_zoo/hailo_models/vehicle_detection/
docker build --build-arg timezone='cat /etc/timezone' -t vehicle_detection:v0 .
```

- This command will build the docker image with the necessary requirements using the Dockerfile that exists in this directory.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all --ipc=host -v /path/to/local/
↪drive:/path/to/docker/dir vehicle_detection:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `vehicle_detection:v0` the name of the docker image.

Finetuning and exporting to ONNX

1. **Train the network on your dataset** Once the docker is started, you can train the vehicle detector on your custom dataset. We recommend following the instructions for YOLOV5 training that can be found in [here](#). The important steps are specified below:

- Update the dataset config file `data/vehicles.yaml` with the paths to your training and validation images files.

```
#update your data paths
train: /path/to/vehicles/training/images/
val: /path/to/vehicles/validation/images/

# number of classes
nc: 1

# class names
names: ['vehicle']
```

- Start training on your dataset starting from our pre-trained weights in `weights/yolov5m_vehicles.pt` (you can also download it from [here](#))

```
python train.py --data ./data/vehicles.yaml --cfg ./models/yolov5m.yaml --
↪weights ./weights/yolov5m_vehicles.pt --epochs 300 --batch 128 --device 1,2,3,
↪4
```

2. **Export to ONNX** Export the model to ONNX using the following command:

```
python models/export.py --weights ./runs/exp<#>/weights/best.pt --img 640 --
↪batch 1
```

- The best model's weights will be saved under the following path: `./runs/exp<#>/weights/best.pt`, where `<#>` is the experiment number.

Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model. In order to do so you need a working model-zoo environment.

Choose the model YAML from our networks configuration directory, i.e.

`hailo_model_zoo/cfg/networks/yolov5m_vehicles.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt yolov5m_vehicles.onnx --calib-path /path/to/calibration/
→ imgs/dir/ --yaml path/to/yolov5m_vehicles.yaml --start-node-names name1 name2 --
→ end-node-names name1 --classes 80
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- `--classes` - adjusting the number of classes in post-processing configuration (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note:

- Since it's an Hailo model, calibration set must be manually supplied.
- This model has an on-chip resize from the video input [1080x1920] to the model's input ([640x640], the resolution the model is trained with). Model Zoo automatically adds the resize for this model using a model script command on `yolov5m_vehicles.alls`. Therefore, the `input_resize` command should be updated if the video input resolution is different (or even removed if it is equal to the resolution the model is trained with).
- On `yolov5m_vehicles.yaml`, change `input_resize` field to match the `input_resize` command on the model script.
- On `yolo.yaml`, change `preprocessing.input_shape` if the network is trained on other resolution.

More details about YAML files are presented [here](#).

5.5.3. Anchors Extraction

The training flow will automatically try to find more fitting anchors values then the default anchors. In our TAPPAS environment we use the default anchors, but you should be aware that the resulted anchors might be different.

The model anchors can be retrieved from the trained model using the following snippet:

```
m = torch.load("last.pt")["model"]
detect = list(m.children())[0][-1]
print(detect.anchor_grid)
```

6. Datasets

The Hailo Model Zoo works with TFRecord files which store the images and labels of the dataset for evaluation and calibration.

The instructions on how to create the TFRecord files are given below. By default, datasets are stored in the following path:

```
~/ .hailomz
```

We recommend to define the data directory path yourself, by setting the HMZ_DATA environment variable.

```
export HMZ_DATA=/new/path/for/storage/
```

- *Datasets*
 - *ImageNet*
 - *COCO2017*
 - *Cityscapes*
 - *WIDERFACE*
 - *VisDrone*
 - *Pascal VOC augmented dataset*
 - *D2S augmented dataset*
 - *NYU Depth V2*
 - *AFLW2k3d and 300W-LP*
 - *Hand Landmark*
 - *Market1501*
 - *PETA*
 - *CelebA*
 - *LFW*
 - *BSD100*
 - **'CIFAR100'**
 - *LOL*
 - *BSD68*
 - *CBSD68*
 - *KITTI_STEREO*
 - *KINETICS400*
 - *NUSCENES*

6.1. ImageNet

To evaluate/optimize/compile the classification models of the Hailo Model Zoo you should generate the ImageNet TFRecord files (manual download is required).

1. Download the ImageNet dataset from [here](#). The expected dataset structure:

```
imagenet
|_ train
|   |_ n01440764
|   |_ ...
|   |_ n15075141
|_ val
|   |_ n01440764
|   |_ ...
|   |_ n15075141
|_ ...
```

* To avoid downloading the ImageNet training data, you may consider using the validation dataset for calibration (does not apply for finetune).

2. Run the create TFRecord scripts:

```
python hailo_model_zoo/datasets/create_imagenet_tfrecord.py val --img /path/
→to/imagenet/val/
python hailo_model_zoo/datasets/create_imagenet_tfrecord.py calib --img /path/
→to/imagenet/train/
```

6.2. COCO2017

To evaluate/optimize/compile the object detection / pose estimation models of the Hailo Model Zoo you should generate the COCO ([link](#)) TFRecord files.

Run the create TFRecord scripts to download the dataset and generate the TFRecord files:

```
python hailo_model_zoo/datasets/create_coco_tfrecord.py val2017
python hailo_model_zoo/datasets/create_coco_tfrecord.py calib2017
```

To evaluate/optimize/compile the single person pose estimation models of the Hailo Model Zoo you should generate the single-person COCO TFRecord files.

Run the create TFRecord scripts to download the dataset and generate the TFRecord files:

```
python hailo_model_zoo/datasets/create_coco_single_person_tfrecord.py val2017
python hailo_model_zoo/datasets/create_coco_single_person_tfrecord.py calib2017
```

6.2.1. Manual Download (Optional)

1. Download COCO ([here](#)). The expected dataset structure:

Annotations:

```
annotations
|_ captions_train2017.json
|_ captions_val2017.json
|_ instances_train2017.json
|_ instances_val2017.json
|_ person_keypoints_train2017.json
|_ person_keypoints_val2017.json
```

Validation set:

```
val2017
|_ 000000000139.jpg
|_ 000000000285.jpg
|_ 000000000632.jpg
|_ 000000000724.jpg
|_ 000000000776.jpg
|_ 000000000785.jpg
|_ 000000000802.jpg
|_ 000000000872.jpg
|_ 000000000885.jpg
|_ ...
```

Training set:

```
train2017
|_ 000000000009.jpg
|_ 000000000025.jpg
|_ 000000000030.jpg
|_ 000000000034.jpg
|_ 000000000036.jpg
|_ 000000000042.jpg
|_ 000000000049.jpg
|_ 000000000061.jpg
|_ 000000000064.jpg
|_ ...
```

2. Run the creation scripts:

```
python hailo_model_zoo/datasets/create_coco_tfrecord.py val2017 --img /path/
→to/val2017 --det /path/to/annotations
python hailo_model_zoo/datasets/create_coco_tfrecord.py calib2017 --img /path/
→to/train2017 --det /path/to/annotations
```

6.3. Cityscapes

To evaluate/optimize/compile the semantic segmentation models of the Hailo Model Zoo you should generate the Cityscapes TFRecord files (manual download is required).

1. Download the Cityscapes dataset from [here](#). The expected dataset structure:

```
Cityscapes
|_ gtFine
|   |_ train
|   |_ test
```

(continues on next page)

(continued from previous page)

```
| | _val
| | _leftImg8bit
| | _train
| | _test
| | _val
| | _train_extra
| _ ...
```

2. Run the create TFRecord scripts:

```
python hailo_model_zoo/datasets/create_cityscapes_tfrecord.py val --data /
↳path/to/Cityscapes/
python hailo_model_zoo/datasets/create_cityscapes_tfrecord.py calib --data /
↳path/to/Cityscapes/
```

6.4. WIDERFACE

To evaluate/optimize/compile the face detection models of the Hailo Model Zoo you should generate the WIDERFACE ([link](#)) TFRecord files.

Run the create TFRecord scripts to download the dataset and generate the TFRecord files:

```
python hailo_model_zoo/datasets/create_widerface_tfrecord.py calib
python hailo_model_zoo/datasets/create_widerface_tfrecord.py val
```

6.4.1. Manual Download (Optional)

1. Download the following from [here](#):
 - WIDER Face Training Images
 - WIDER Face Validation Images
 - Face annotations
2. Download the following from [here](#)
 - [wider_hard_val.mat](#)

Expected directory structure:

```
widerface/
| _wider_face_split
| | _readme.txt
| | _wider_face_test_filelist.txt
| | _wider_face_test.mat
| | _wider_face_train_bbx_gt.txt
| | _wider_face_train.mat
| | _wider_face_val_bbx_gt.txt
| | _wider_face_val.mat
| | _wider_hard_val.mat
| _WIDER_train
| | _images
| | | _0--Parade
| | | _10--People_Marching
| | | _11--Meeting
| | | _ ...
| _WIDER_val
```

(continues on next page)

(continued from previous page)

```
|_ images
|_ 0--Parade
|_ 10--People_Marching
|_ 11--Meeting
|_ ...
```

3. Run the creation scripts

```
python hailo_model_zoo/datasets/create_widerface_tfrecord.py calib --img /
→path/to/widerface --gt_mat_path /path/to/wider_face_split --hard_mat_path /
→path/to/wider_face_split
python hailo_model_zoo/datasets/create_widerface_tfrecord.py val --img /path/
→to/widerface --gt_mat_path /path/to/wider_face_split --hard_mat_path /path/
→to/wider_face_split
```

6.5. VisDrone

To evaluate/optimize/compile the visdrone object detection models of the Hailo Model Zoo you should generate the VisDrone ([link](#)) TFRecord files.

Run the create TFRecord scripts to download the dataset and generate the TFRecord files:

```
python hailo_model_zoo/datasets/create_visdrone_tfrecord.py train
python hailo_model_zoo/datasets/create_visdrone_tfrecord.py val
```

6.5.1. Manual Download (Optional)

1. Download VisDrone ([here](#)). The expected dataset structure:

Training set:

```
VisDrone2019-DET-train/
|_ annotations
| |_ 0000002_00005_d_0000014.txt
| |_ 0000002_00448_d_0000015.txt
| |_ ...
|_ images
| |_ 0000002_00005_d_0000014.jpg
| |_ 0000002_00448_d_0000015.jpg
| |_ ...
```

Validation set:

```
VisDrone2019-DET-val/
|_ annotations
| |_ 0000001_02999_d_0000005.txt
| |_ 0000001_03499_d_0000006.txt
| |_ ...
|_ images
| |_ 0000001_02999_d_0000005.jpg
| |_ 0000001_03499_d_0000006.jpg
| |_ ...
```

2. Run the creation scripts:

```
python hailo_model_zoo/datasets/create_visdrone_tfrecord.py train -d /path/to/
↳VisDrone2019-DET-train
python hailo_model_zoo/datasets/create_visdrone_tfrecord.py val -d /path/to/
↳VisDrone2019-DET-val
```

6.6. Pascal VOC augmented dataset

Run the creation scripts:

```
python hailo_model_zoo/datasets/create_pascal_tfrecord.py calib
python hailo_model_zoo/datasets/create_pascal_tfrecord.py val
```

6.6.1. Manual Download (Optional)

1. Download the dataset from [here](#). Expected dataset structure:

```
benchmark_RELEASE
|_ dataset
|_ cls
|_ img
|_ inst
|_ train.txt
|_ val.txt
```

2. run the creation scripts:

```
python hailo_model_zoo/datasets/create_pascal_tfrecord.py calib --root
↳benchmark_RELEASE/dataset
python hailo_model_zoo/datasets/create_pascal_tfrecord.py val --root
↳benchmark_RELEASE/dataset
```

6.7. D2S augmented dataset

Run the creation scripts:

```
python hailo_model_zoo/datasets/create_d2s_tfrecord.py calib
python hailo_model_zoo/datasets/create_d2s_tfrecord.py val
```

6.7.1. Manual Download (Optional)

1. Download the dataset from [here](#). Extract using 'tar -xf d2s_images_v1.1.tar.xz'. Expected dataset structure:

```
|_ images
|_ D2S_000200.jpg
|_ D2S_000201.jpg
|_ ...
```

2. Download the annotations from [here](#). Extract using 'tar -xf d2s_annotations_v1.1.tar.xz'. Expected annotations structure:

```
|_ annotations
|_ D2S_augmented.json
|_ D2S_validation.json
|_ ...
```

- run the creation scripts:

```
python hailo_model_zoo/datasets/create_d2s_tfrecord.py calib --img /path/to/
→dataset --det /path/to/annotations/D2S_augmented.json
python hailo_model_zoo/datasets/create_d2s_tfrecord.py val --img /path/to/
→dataset --det /path/to/annotations/D2S_validation.json
```

6.8. NYU Depth V2

Run the creation scripts:

```
python hailo_model_zoo/datasets/create_nyu_depth_v2_tfrecord.py calib
python hailo_model_zoo/datasets/create_nyu_depth_v2_tfrecord.py val
```

6.8.1. Manual Download (Optional)

- Download the dataset from [here](#). Extract using 'tar -xf nyudepthv2.tar.gz'. Expected dataset structure:

```
|_ train
|_ study_0300
|_ 00626.h5
|_ 00631.h5
|_ ...
|_ ...
|_ val
|_ official
|_ 00001.h5
|_ 00002.h5
|_ 00009.h5
|_ 00014.h5
|_ ...
```

- run the creation scripts:

```
python hailo_model_zoo/datasets/create_nyu_depth_v2_tfrecord.py calib --data .
→/nyu_depth_v2/
python hailo_model_zoo/datasets/create_nyu_depth_v2_tfrecord.py val --data ./
→nyu_depth_v2/
```

6.9. AFLW2k3d and 300W-LP

Run the creation scripts:

```
python hailo_model_zoo/datasets/create_300w_lp_tddfa_tfrecord.py
python hailo_model_zoo/datasets/create_aflw2k3d_tddfa_tfrecord.py
```

6.9.1. Manual Download (Optional)

1. Download the augmented_cropped 300W-LP dataset from [here](#) and extract. Expected structure:

```
train_aug_120x120
|_ AFW_AFW_1051618982_1_0_10.jpg
|_ AFW_AFW_1051618982_1_0_11.jpg
|_ AFW_AFW_1051618982_1_0_12.jpg
|_ AFW_AFW_1051618982_1_0_13.jpg
|_ AFW_AFW_1051618982_1_0_1.jpg
|_ AFW_AFW_1051618982_1_0_2.jpg
|_ AFW_AFW_1051618982_1_0_3.jpg
|_ AFW_AFW_1051618982_1_0_4.jpg
|_ ...
```

2. Run

```
python hailo_model_zoo/datasets/create_300w-lp_tddfa_tfrecord.py --dir /path/
→to/train_aug_120x120
```

3. Download the following files:

- the official dataset from [here](#)
- the cropped dataset from [here](#)
- The following files from [here](#)
 - AFLW2000-3D.pose.npy
 - AFLW2000-3D.pts68.npy
 - AFLW2000-3D-Reannotated.pts68.npy
 - AFLW2000-3D_crop.roi_box.npy

The expected structure:

```
aflw2k3d_tddfa
|_ AFLW2000-3D_crop.roi_box.npy
|_ AFLW2000-3D.pose.npy
|_ AFLW2000-3D.pts68.npy
|_ AFLW2000-3D-Reannotated.pts68.npy
|_ test.data
|_ AFLW2000
|   |_ Code
|   | |_ Mex
|   | |_ ModelGeneration
|   | |_ image00002.jpg
|   | |_ image00002.mat
|   | |_ image00004.jpg
|   | |_ image00004.mat
|   | |_ ...
|_ AFLW2000-3D_crop
|   |_ image00002.jpg
|   |_ image00004.jpg
|   |_ image00006.jpg
|   |_ image00008.jpg
|   |_ ...
|_ AFLW2000-3D_crop.list
|_ AFLW_GT_crop
|   |_ ...
|_ AFLW_GT_crop.list
```

4. Run the following:

```
python hailo_model_zoo/datasets/create_aflw2k3d_tddfa_tfrecord.py --dir /path/
→to/aflw2k3d_tddfa
```

- BFM files are removed from face_landmarks_3d_postprocessing. From now on, users must download them manually. For more information, please refer to: https://github.com/cleardusk/3DDFA_V2/tree/master/bfm

6.10. Hand Landmark

Run the creation script:

```
python hailo_model_zoo/datasets/create_hand_landmark_tfrecord.py
```

6.10.1. Manual Download (Optional)

1. Download the dataset from [here](#) and extract. Expected structure:

```
Hands      00 000
|_ Hand_0011695.jpg
|_ Hand_0011696.jpg
|_ Hand_0011697.jpg
|_ ...
```

2. Run

```
python hailo_model_zoo/datasets/create_hand_landmark_tfrecord.py --img /path/
→to/Hands
```

6.11. Market1501

Run the creation scripts:

```
python hailo_model_zoo/datasets/create_market_tfrecord.py val
python hailo_model_zoo/datasets/create_market_tfrecord.py calib
```

6.11.1. Manual Download (Optional)

1. Download the dataset from [here](#) and extract.

Expected structure:

```
Market-1501-v15.09.15
|_ bounding_box_test
|_ 0000_c1s1_000151_01.jpg
|_ 0000_c1s1_000376_03.jpg
|_ ...
|_ bounding_box_train
|_ 0002_c1s1_000451_03.jpg
|_ 0002_c1s1_000551_01.jpg
|_ ...
|_ gt_bbox
|_ 0001_c1s1_001051_00.jpg
|_ 0001_c1s1_002301_00.jpg
|_ ...
|_ gt_query
```

(continues on next page)

(continued from previous page)

```
|_ 0001_c1s1_001051_00_good.mat
|_ 0001_c1s1_001051_00_junk.mat
|_ ...
|_ query
|_ 0001_c1s1_001051_00.jpg
|_ 0001_c2s1_000301_00.jpg
|_ ...
```

2. Run

```
python hailo_model_zoo/datasets/create_market_tfrecord.py val --img path/to/
↳Market-1501-v15.09.15/
python hailo_model_zoo/datasets/create_market_tfrecord.py calib --img path/to/
↳Market-1501-v15.09.15/bounding_box_train/
```

6.12. PETA

To evaluate/optimize/compile the person attribute models of the Hailo Model Zoo you should generate the PETA TFRecord files (manual download is required).

1. Download the PETA dataset from [here](#). The expected dataset structure:

```
PETA
|_ images
| |_ 00001.png
| |_ ...
| |_ 19000.png
|_ PETA.mat
```

2. Run the create TFRecord scripts:

```
python hailo_model_zoo/datasets/create_peta_tfrecord.py test --data /path/to/
↳PETA/
python hailo_model_zoo/datasets/create_peta_tfrecord.py train --data /path/to/
↳PETA/
```

6.13. CelebA

To evaluate/optimize/compile the face attribute models of the Hailo Model Zoo you should generate the CelebA TFRecord files (manual download is required).

1. Download the CelebA dataset from [here](#). The expected dataset structure:

```
Celeba
|_ img_align_celeba_png
| |_ 000001.jpg
| |_ ...
| |_ 202599.jpg
|_ list_attr_celeba.txt
|_ list_eval_partition.txt
```

2. Run the create TFRecord scripts:

```
python hailo_model_zoo/datasets/create_celeba_tfrecord.py val --data /path/to/
↳CelebA/
python hailo_model_zoo/datasets/create_celeba_tfrecord.py train --data /path/
↳to/CelebA/
```

6.14. LFW

To evaluate/optimize/compile the face recognition models of the Hailo Model Zoo you should generate the arcface_lfw TFRecord files

Run the creation scripts:

```
python hailo_model_zoo/datasets/create_arcface_lfw_tfrecord.py calib -
python hailo_model_zoo/datasets/create_arcface_lfw_tfrecord.py val
```

6.14.1. Manual Download (Optional)

1. Download LFW dataset from [here](#)
2. Download LFW pairs file from [here](#)
3. Run the scripts:

```
python hailo_model_zoo/datasets/create_arcface_lfw_tfrecord.py calib -
  ↳ -tgz /path/to/lfw.tgz --pairs /path/to/pairs.txt
python hailo_model_zoo/datasets/create_arcface_lfw_tfrecord.py val --
  ↳ -tgz /path/to/lfw.tgz --pairs /path/to/pairs.txt
```

6.15. BSD100

To evaluate/optimize/compile the super resolution models of the Hailo Model Zoo you should generate the BSD100 TFRecord files.

Run the creation scripts:

```
python hailo_model_zoo/datasets/create_bsd100_tfrecord.py val
python hailo_model_zoo/datasets/create_bsd100_tfrecord.py calib
```

6.15.1. Manual Download (Optional)

1. Download the BSD100 dataset from [here](#) and extract. The expected dataset structure:

```
BSD100
|_ GTmod12
| |_ 101085.png
| |_ ...
| |_ 97033.png
|_ GTmod16
| |_ ...
|_ LRbicx8
| |_ ...
|_ LRbicx4
| |_ ...
|_ LRbicx3
| |_ ...
|_ LRbicx2
| |_ ...
|_ LRbicx16
| |_ ...
|_ original
| |_ ...
```

2. Run the scripts:

```
python hailo_model_zoo/datasets/create_bsd100_tfrecord.py val --lr /path/to/
↳LRbicx4 --hr /path/to/GTmod12
python hailo_model_zoo/datasets/create_bsd100_tfrecord.py calib --lr /path/to/
↳LRbicx4 --hr /path/to/GTmod12
```

6.16. CLIP_CIFAR100

To evaluate/optimize/compile the CLIP models of the Hailo Model Zoo you should generate the CIFAR100 TFRecord files.

Run the creation scripts:

```
python hailo_model_zoo/datasets/create_clip_cifar100_tfrecord.py val
python hailo_model_zoo/datasets/create_clip_cifar100_tfrecord.py calib
```

6.17. LOL

To evaluate/optimize/compile the low light enhancement models of the Hailo Model Zoo you should generate the LOL TFRecord files.

Run the creation scripts:

```
python hailo_model_zoo/datasets/create_lol_tfrecord.py val
python hailo_model_zoo/datasets/create_lol_tfrecord.py calib
```

6.17.1. Manual Download (Optional)

1. Download the LOL dataset from [here](#) and extract. The expected dataset structure:

```
lol_dataset
|_ eval15
|   |_ high
|       |_ 111.png
|       |_ 146.png
|       |_ ...
|   |_ low
|       |_ 111.png
|       |_ 146.png
|       |_ ...
|_ our485
|   |_ high
|       |_ 100.png
|       |_ 101.png
|       |_ ...
|   |_ low
|       |_ 100.png
|       |_ 101.png
|       |_ ...
```

2. Run the scripts:

```
python hailo_model_zoo/datasets/create_lol_tfrecord.py val --ll /path/to/val/
↳lowlight/images --lle /path/to/val/highlight/images
python hailo_model_zoo/datasets/create_lol_tfrecord.py calib --ll /path/to/
↳train/lowlight/images --lle /path/to/train/highlight/images
```

6.18. BSD68

To evaluate/optimize/compile the image denoising models of the Hailo Model Zoo you should generate the BSD68 TFRecord files.

Run the creation scripts:

```
python hailo_model_zoo/datasets/create_bsd68_tfrecord.py val
python hailo_model_zoo/datasets/create_bsd68_tfrecord.py calib
```

6.18.1. Manual Download (Optional)

1. Download the BSD100 dataset from [here](#) and extract. The expected dataset structure:

```
test
|_ BSD68
| |_ test001.png
| |_ ...
| |_ test068.png
|_ CBSD68
| |_ ...
|_ Kodak
| |_ ...
|_ McMaster
| |_ ...
|_ Set12
| |_ ...
|_ Urban100
| |_ ...
|_ LRBicx16
```

2. Run the scripts:

```
python hailo_model_zoo/datasets/create_bsd100_tfrecord.py BSD68 val --data-
→path <BSD68-extracted-data-folder>
python hailo_model_zoo/datasets/create_bsd100_tfrecord.py BSD68 calib --data-
→path <BSD68-extracted-data-folder>
```

6.19. CBSD68

To evaluate/optimize/compile the image denoising models of the Hailo Model Zoo you should generate the CBSD68 TFRecord files.

Run the creation scripts:

```
python hailo_model_zoo/datasets/create_bsd68_tfrecord.py CBSD68 val
python hailo_model_zoo/datasets/create_bsd68_tfrecord.py CBSD68 calib
```

6.19.1. Manual Download (Optional)

1. Download the BSD100 dataset from [here](#) and extract. The expected dataset structure:

```
test
|_ BSD68
| |_ ...
|_ CBSD68
| |_ test001.png
| |_ ...
| |_ test068.png
|_ Kodak
| |_ ...
|_ McMaster
| |_ ...
|_ Set12
| |_ ...
|_ Urban100
| |_ ...
|_ LRBicx16
```

2. Run the scripts:

```
python hailo_model_zoo/datasets/create_bsd100_tfrecord.py CBSD68 val --data-
→path <CBSD68-extracted-data-folder>
python hailo_model_zoo/datasets/create_bsd100_tfrecord.py CBSD68 calib --data-
→path <CBSD68-extracted-data-folder>
```

6.20. KITTI_STEREO

To evaluate/optimize/compile the stereo models of the Hailo Model Zoo you should generate the KITTI Stereo TFRecord files.

6.20.1. Manual Download

1. Download the KITTI Stereo dataset from [here](#). One must request access and await approval.
2. Extract the dataset. The expected dataset structure:

```
kitti_stereo
|_ testing
| |_ image_2
| | |_ 000000_10.png
| | |_ 000000_11.png
| | |_ ...
| |_ image_3
| | |_ 000000_10.png
| | |_ 000000_11.png
| | |_ ...
|_ training
| |_ image_2
| | |_ 000000_10.png
| | |_ 000000_11.png
| | |_ ...
| |_ disp_occ_0
| | |_ 000000_10.png
| | |_ 000001_10.png
| | |_ 000002_10.png
| | |_ ...
```

- Run the scripts:

```
python hailo_model_zoo/datasets/create_kitti_stereo_tfrecord.py calib --data
→ <TRAIN_DIR>
python hailo_model_zoo/datasets/create_kitti_stereo_tfrecord.py val --data
→ <VALIDATION_DIR>
```

6.21. KINETICS400

To evaluate/optimize/compile the video classification models of the Hailo Model Zoo you should generate the KINETICS400 TFRecord files.

6.21.1. Manual Download

- Download the kinetics400 dataset from [here](#). Follow the instructions to download the dataset.
- The expected dataset structure: .. code-block:

```
k400/videos
|_ test
|   |_ abseiling
|       |_ 0aSqlZT8QmM_000048_000058.mp4
|       |_ 0xreS8KFbrw_000417_000427.mp4
|       |_ ...
|   |_ air drumming
|       |_ 013SMb0SX8I_000020_000030.mp4
|       |_ 013SMb0SX8I_000020_000030.mp4
|       |_ ...
|_ train
|   |_ abseiling
|       |_ 0347ZoDXyP0_000095_000105.mp4
|       |_ 035LtPeUFTE_000085_000095.mp4
|       |_ ...
|   |_ air drumming
|       |_ 03V2idM7_KY_000003_000013.mp4
|       |_ 1R7Ds_000003_000013.mp4
|       |_ 0c1bhfxioqE_000078_000088.mp4
|       |_ ...
|_ val
|   |_ abseiling
|       |_ 0wR5jVB-WPk_000417_000427.mp4
|       |_ 3caPS4FHFF8_000036_000046.mp4
|       |_ ...
|   |_ air drumming
|       |_ 2cPLjY5AWXU_000001_000011.mp4
|       |_ 3K0Sw7rbzPU_000114_000124.mp4
|       |_ 6Tnsmk9C2rg_000048_000058.mp4
|       |_ ...
```

- Run the scripts:

```
python hailo_model_zoo/datasets/create_kinetics400_tfrecord.py calib --data
→ <path_to_k400/videos>
python hailo_model_zoo/datasets/create_kinetics400_tfrecord.py val --data
→ <path_to_k400/videos>
```

6.22. NUSCENES

1. Download the dataset from [here](#) and extract.
2. The expected dataset structure:

```
nuscenes
|_ maps
|   |_ *.png
|_ samples
|   |_ CAM_BACK
|       |_ *.jpg
|       |_ CAM_BACK_LEFT
|           |_ *.jpg
|       |_ CAM_BACK_RIGHT
|           |_ *.jpg
|       |_ CAM_FRONT
|           |_ *.jpg
|       |_ CAM_FRONT_LEFT
|           |_ *.jpg
|       |_ CAM_FRONT_RIGHT
|           |_ *.jpg
|_ sweeps
|   |_ CAM_BACK
|       |_ *.jpg
|       |_ CAM_BACK_LEFT
|           |_ *.jpg
|       |_ CAM_BACK_RIGHT
|           |_ *.jpg
|       |_ CAM_FRONT
|           |_ *.jpg
|       |_ CAM_FRONT_LEFT
|           |_ *.jpg
|       |_ CAM_FRONT_RIGHT
|           |_ *.jpg
|_ v1.0-trainval
|   |_ *.json
```

3. Run the scripts:

```
python hailo_model_zoo/datasets/create_nuscenes_petr2_cascade_tfrecord.py
→calib --ann_file <train_annotation_file.pkl> --coords-dir <coords3d_
→directory_path>
python hailo_model_zoo/datasets/create_nuscenes_petr2_cascade_tfrecord.py
→val --ann_file <val_annotation_file.pkl> --coords-dir <coords3d_directory_
→path>
```

Where <*_annotation_file.pkl> is the train / val .pkl annotation file generated from the [PETR training environment](#).

Notice: In order to benchmark our PETRv2 cascade (petrv2), please download the annotation .pkl file from [here](#) and create a symbolic link (softlink) from `/fastdata/data/nuscenes/nuesence/` to your nuscenes dataset folder.

7. Benchmarks

In order to measure FPS, power and latency of the Hailo Model Zoo networks you can use the HailoRT command line interface.

For more information please refer to the HailoRT documentation in hailo.ai.

7.1. Example

The HailoRT command line interface works with the Hailo Executable File (HEF) of the model.

To generate the HEF file use the following command:

```
hailomz compile <model_name>
```

After building the HEF you will be able to measure the performance of the model by using the HailoRT command line interface.

Example for measuring performance of resnet_v1_50:

```
hailortcli benchmark resnet_v1_50.hef
```

Example output:

```
=====
Summary
=====
FPS   (hw_only)           = 1328.83
      (streaming)        = 1328.8
Latency (hw)             = 2.93646 ms
Power in streaming mode (average) = 3.19395 W
      (max)              = 3.20456 W
```

7.2. Using Datasets from the Hailo Model Zoo

To use datasets from the Hailo Model Zoo, you can use the command:

```
python hailo_model_zoo/tools/conversion_tool.py /path/to/tfrecord_file resnet_v1_
↪ 50
```

which will generate a bin file with serialized images. This bin file can be used inside the HailoRT:

```
hailortcli benchmark resnet_v1_50.hef --input-files tfrecord_file.bin
```


8. Hailo Model Zoo YAML Description

8.1. Properties

- **network**
 - **network_name** (*['string', 'null']*): The network name. Default: `None`.
- **paths**
 - **network_path** (*array*): Path to the network files, can be ONNX / TF Frozen graph (pb) / TF CKPT files / TF2 saved model.
 - **alls_script** (*['string', 'null']*): Path to model script in the alls directory.
- **parser**
 - **nodes** (*['array', 'null']*): List of [start node, [end nodes]] for parsing.
For example: ["resnet_v1_50/conv1/Pad", "resnet_v1_50/predictions/Softmax"].
 - **normalization_params**: Add normalization on chip.
For example: `normalization_params: { "normalize_in_net": true, "mean_list": [123.68, 116.78, 103.94], "std_list": [58.395, 57.12, 57.375] }`.
 - * **normalize_in_net** (*boolean*): Whether or not the network includes an on-chip normalization layer. If so, the normalization layer will appear on the .alls file that is used. Default: `False`.
Example alls command: `normalization1 = normalization([123.675, 116.28, 103.53], [58.395, 57.12, 57.375])`
If the alls doesn't include the required normalization, then the MZ (and the user application) will apply normalization before feeding inputs to the network
 - * **mean_list** (*['array', 'null']*): Used only in case `normalize_in_net=false`. The MZ automatically performs normalization to the calibration set, so the network receives already-normalized input (saves the user the need to normalize the dataset). Default: `None`.
 - * **std_list** (*['array', 'null']*): Used only in case `normalize_in_net=false`. The MZ automatically performs normalization to the calibration set, so the network receives already-normalized input (saves the user the need to normalize the dataset). Default: `None`.
 - **start_node_shapes** (*['array', 'null']*): Dict for setting input shape of supported models that does not explicitly use it.
For example, models with input shape of [?, ?, ?, 3] can be set with {"Preprocessor/sub:0": [1, 512, 512, 3]}. Default: `None`.
- **preprocessing**
 - **network_type** (*['string', 'null']*): The type of the network. Default: `classification`.
 - **meta_arch** (*['string', 'null']*): The network preprocessing meta-architecture.
For example: `mobilenet_ssd`, `efficientnet`, etc. Default: `None`.
 - **padding_color** (*['integer', 'null']*): On the training environments, the input images to the model have used this color to indicate "padding" around resized images. Default: `114` for YOLO architectures, `0` for others.
- **quantization**
 - **calib_set** (*['array', 'null']*): List contains the calibration set path.
For example: ["models_files/imagenet/2021-06-20/imagenet_calib.tfrecord"]. Default: `None`.
 - **calib_set_name** (*['string', 'null']*): Name of the dataset used for calibration. By default (null) uses the evaluation dataset name. Default: `None`.
- **postprocessing**
 - **meta_arch** (*['string', 'null']*): Postprocessing meta architecture name.

For example: yolo_v3, yolo_v4, etc. Default: `None`.

- **postprocess_config_file** (*[string, 'null']*): Path to a file with the postprocessing configuration (for example, for offloading NMS to the Hailo-8). Default: `None`.
- **hpp** (*boolean*): Activate the Hailo postprocessing on host. Default: `False`.
- **device_pre_post_layers**: Whether to use postprocessing on chip or do it on the host.
 - * **bilinear** (*boolean*): Activate the bilinear PPU layer. Default: `False`.
 - * **argmax** (*boolean*): Activate the Argmax PPU layer. Default: `False`.
 - * **softmax** (*boolean*): Activate the Softmax PPU layer. Default: `False`.
 - * **nms** (*boolean*): Activate the NMS PPU layer and the relevant decoding layers. Default: `False`.
- **evaluation**
 - **dataset_name** (*[string, 'null']*): Name of the dataset to be used in evaluation. Default: `None`.
 - **data_set** (*[string, 'null']*): Path to TFrecord dataset for evaluation. Default: `None`.
 - **classes** (*[integer, 'null']*): Number of classes in the model. Default: `1000`.
 - **labels_offset** (*[integer, 'null']*): Offset of labels. Default: `0`.
 - **network_type** (*[string, 'null']*): The type of the network used for evaluation.
Use this field if evaluation type is different than preprocessing type. Default: `None`.
- **hn_editor**
 - **yuv2rgb** (*boolean*): Add YUV to RGB layer. Default: `False`.
 - **flip** (*boolean*): Rotate input by 90 degrees. Default: `False`.
 - **input_resize**: Add resize bilinear layer at the start of the network.
 - * **enabled** (*boolean*): Whether this is enabled or disabled. Default: `False`.
 - * **input_shape** (*array*): List of input shape to resize from [H, W].
 - **bgr2rgb** (*boolean*): Add BGR to RGB layer.
On some training frameworks, the models are trained on BGR inputs. When we want to feed RGB images to the network (whether on the MZ or on the user application), we need to transform the images from RGB to BGR. The MZ automatically inserts this layer to the on-chip model.
We have already set the "bgr2rgb" flag on the yaml files that correspond to the relevant retraining dockers. Default: `False`.

8.2. YAML hierarchies

- **The MZ uses hierarchical .yaml infrastructure for configuration. For example, for yolov5m_vehicles:**
 - Network yaml is `networks/yolov5m_vehicles.yaml`
 - It includes at the top the lines:


```
base:
- base/yolov5.yaml
```
 - Meaning it inherits from `base/yolov5.yaml`
 - Which inherits from `base/yolo.yaml`
 - Which inherits from `base/base.yaml`
- Each property on the child hierarchies replaces the properties on the parent ones. For example, if `preprocessing.input_shape` is defined both in `base/yolov5.yaml` and `base/base.yaml`, the one from `base/yolov5.yaml` will be used

- Therefore, if we want to change some property, we can just update the last child file that is using that property

8.3. Notes for Retraining

- `evaluation` and `postprocessing` properties aren't needed for compilation as they are used by the Model-Zoo for model evaluation (which isn't supported yet for retrained models). Also `info` field is just used for description.
 - Only on YOLOv4 family, the `evaluation.classes` and `postprocessing.anchors.sizes` fields are used for compilation, that's why you should update those values even if just for compilation
- You might want to update those default values on some advanced scenarios:
 - `preprocessing.padding_color`
 - ✳ Change those values only if you have used a different value for training your model
 - `parser.normalization_params.normalize_in_net`
 - ✳ If you have manually changed the normalization values on the retraining docker, and *normalize_in_net=true*, remember to update the corresponding `alls` command
 - `parser.normalization_params.mean_list`
 - ✳ Update those values if *normalize_in_net=false* and you have manually changed the normalization values on the retraining docker
 - `parser.normalization_params.std_list`
 - ✳ Update those values if *normalize_in_net=false* and you have manually changed the normalization values on the retraining docker

9. Retrain on Custom Dataset

This page will help you ramping-up your training environment for various tasks and architectures. Each architecture has its own README which describes how to:

1. Setup the environment using a compatible Dockerfile.
2. Start the training process.
3. Export your model.

Important: Retraining is not available inside the docker version of Hailo Software Suite. In case you use it, clone the hailo_model_zoo outside of the docker, and perform the retraining there: `git clone https://github.com/hailo-ai/hailo_model_zoo.git`

Object Detection

- [YOLOv3](#)
- [YOLOv4](#)
- [YOLOv5](#)
- [YOLOv8](#)
- [YOLOX](#)
- [DAMO-YOLO](#)
- [NanoDet](#)

Pose Estimation

- [CenterPose](#)

Single Person Pose Estimation

- [MSPN](#)

Semantic Segmentation

- [FCN](#)

Instance Segmentation

- [YOLACT](#)
- [YOLOv8_seg](#)

Face Recognition

- [ArcFace](#)

9.1. YOLOv3 Retraining

- To learn more about yolov3 look [here](#)
-

9.1.1. Prerequisites

- [docker \(installation instructions\)](#)
- [nvidia-docker2 \(installation instructions\)](#)

NOTE:In case you use Hailo Software Suite docker, make sure you are doing all the following instructions outside of this docker.

9.1.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/yolov3
docker build --build-arg timezone='cat /etc/timezone' -t yolov3:v0 .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.
- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.
- This command will build the docker image with the necessary requirements using the Dockerfile exists in this directory.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all --ipc=host -v /path/to/local/
→data/dir:/path/to/docker/data/dir yolov3:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `yolov3:v0` the name of the docker image.

9.1.3. Training and exporting to ONNX

1. Train your model: Once the docker is started, you can start training your YOLOv3.

- Prepare your custom dataset - Follow the full instructions described [here](#):
 - Create `data/obj.data` with paths to your training and validation `.txt` files, which contain the list of the image paths*.

```
classes = 80
train = data/train.txt
valid = data/val.txt
names = data/coco.names
backup = backup/
```

* Tip: specify the paths to the training and validation images in the training and validation `.txt` files relative to `/workspace/darknet/`

Place your training/validation images and labels in your data folder and make sure you update the number of classes.

- Labels - each image should have labels in YOLO format with corresponding txt file for each image.
- Start training - The following command is an example for training the yolov3.

```
./darknet detector train data/obj.data cfg/yolov3.cfg yolov3.weights -map -clear
```

Final trained weights will be available in `backup/` directory.

2. Export to ONNX:

In order to export your trained YOLOv3 model to ONNX run the following script:

```
python ./pytorch-YOLOv4/demo_darknet2onnx.py cfg/yolov3.cfg /path/to/trained.
→weights /path/to/some/image.jpg 1
```

- The ONNX would be available in `/workspace/darknet/`

9.1.4. Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model.

In order to do so you need a working model-zoo environment.

Choose the corresponding YAML from our networks configuration directory, i.e.

`hailo_model_zoo/cfg/networks/yolov3_416.yaml` (for the default YOLOv3 model).

Align the corresponding alls, i.e. `hailo_model_zoo/cfg/alls/base/yolov3_416.alls` with the size of the calibration set using `dataset_size=<number_of_jpgs_in_folder>` parameter.

Run compilation using the model zoo:

```
hailomz compile yolov3_416 --ckpt yolov3_1_416_416.onnx --calib-path /path/to/
↳calibration/imgs/dir/
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- The model zoo will take care of adding the input normalization to be part of the model.

Note:

- On your desired YOLOv3 YAML, make sure `preprocessing.input_shape` fits your model's resolution.
- For TAPPAS, retrain the model with a resolution of 608x608, and on compilation use `yolov3_gluon.yaml`.

More details about YAML files are presented [here](#).

9.2. YOLOv4-leaky Retraining

- To learn more about yolov4 look [here](#)

9.2.1. Prerequisites

- docker ([installation instructions](#))
- nvidia-docker2 ([installation instructions](#))

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

9.2.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/yolov4
docker build --build-arg timezone='cat /etc/timezone' -t yolov4:v0 .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.
- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.
- This command will build the docker image with the necessary requirements using the Dockerfile exists in this directory.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all --ipc=host -v /path/to/local/
↳data/dir:/path/to/docker/data/dir yolov4:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `yolov4:v0` the name of the docker image.

9.2.3. Training and exporting to ONNX

1. Train your model:

Once the docker is started, you can start training your YOLOv4-leaky.

- Prepare your custom dataset - Follow the full instructions described [here](#):
 - Create `data/obj.data` with paths to your training and validation `.txt` files, which contain the list of the image paths*.

```
classes = 80
train = data/train.txt
valid = data/val.txt
names = data/coco.names
backup = backup/
```

* Tip: specify the paths to the training and validation images in the training and validation `.txt` files relative to `/workspace/darknet/`

Place your training/validation images and labels in your data folder and make sure you update the number of classes.

- Labels - each image should have labels in YOLO format with corresponding txt file for each image.
- Start training - The following command is an example for training the yolov4-leaky model.

```
./darknet detector train data/obj.data cfg/yolov4-leaky.cfg -map -clear
```

Final trained weights will be available in `backup/` directory.

2. Export to ONNX:

In order to export your trained YOLOv4 model to ONNX run the following script:

```
python ../pytorch-YOLOv4/demo_darknet2onnx.py cfg/yolov4-leaky.cfg /path/to/
→trained.weights /path/to/some/image.jpg 1
```


- The ONNX will be available in `/workspace/darknet/`

Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model.

In order to do so you need a working model-zoo environment.

Choose the corresponding YAML from our networks configuration directory, i.e.

`hailo_model_zoo/cfg/networks/yolov4_leaky.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt yolov4_1_3_512_512.onnx --calib-path /path/to/calibration/
→ imgs/ --yaml path/to/yolov4_leaky.yaml --start-node-names name1 name2 --end-node-
→ names name1
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note:

- On your desired YOLOv4 YAML, update `postprocessing.anchors.sizes` property if anchors changed, and `preprocessing.input_shape` if the network is trained on other resolution.
- On `yolo.yaml`, change `evaluation.classes` if classes amount is changed.

More details about YAML files are presented [here](#).

9.3. YOLOv5 Retraining

- To learn more about yolov5 look [here](#)

9.3.1. Prerequisites

- docker ([installation instructions](#))
- nvidia-docker2 ([installation instructions](#))

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

9.3.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/yolov5
docker build --build-arg timezone='cat /etc/timezone' -t yolov5:v0 .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.
- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.

* This command will build the docker image with the necessary requirements using the Dockerfile exists in yolov5 directory.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all --ipc=host -v /path/to/
→local/data/dir:/path/to/docker/data/dir yolov5:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `yolov5:v0` the name of the docker image.

9.3.3. Training and exporting to ONNX

1. Train your model:

Once the docker is started, you can start training your model.

- Prepare your custom dataset - Follow the steps described [here](#) in order to create:
 - `dataset.yaml` configuration file
 - Labels - each image should have labels in YOLO format with corresponding txt file for each image.
 - Make sure to include number of classes field in the yaml, for example: `nc: 80`
- Start training - The following command is an example for training a `yolov5s` model.

```
python train.py --img 640 --batch 16 --epochs 3 --data coco128.yaml --weights
→yolov5s.pt --cfg models/yolov5s.yaml
```

- `yolov5s.pt` - pretrained weights. You can find the pretrained weights for `yolov5s`, `yolov5m`, `yolov5l`, `yolov5x` and `yolov5m_wo_spp` in your working directory.
- `models/yolov5s.yaml` - configuration file of the yolov5 variant you would like to train. In order to change the number of classes make sure you update this file.

NOTE: We recommend to use `yolov5m_wo_spp` for best performance on Hailo-8

2. Export to ONNX:

In order to export your trained YOLOv5 model to ONNX run the following script:

```
python models/export.py --weights /path/to/trained/model.pt --img 640 --batch 1
→ # export at 640x640 with batch size 1
```

9.3.4. Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model.

In order to do so you need a working model-zoo environment.

Choose the corresponding YAML from our networks configuration directory, i.e.

`hailo_model_zoo/cfg/networks/yolov5s.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt yolov5s.onnx --calib-path /path/to/calibration/imgs/dir/ --
→yaml path/to/yolov5s.yaml --start-node-names name1 name2 --end-node-names name1 --
→classes 80
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- `--classes` - adjusting the number of classes in post-processing configuration (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note:

- Make sure to also update `preprocessing.input_shape` field on `yolo.yaml`, if it was changed on re-training.

More details about YAML files are presented [here](#).

9.3.5. Anchors Extraction

The training flow will automatically try to find more fitting anchors values then the default anchors. In our TAPPAS environment we use the default anchors, but you should be aware that the resulted anchors might be different. The model anchors can be retrieved from the trained model using the following snippet:

```
m = torch.load("last.pt")["model"]
detect = list(m.children())[0][-1]
print(detect.anchor_grid)
```

9.4. YOLOv8 Retraining

- To learn more about yolov8 look [here](#)

9.4.1. Prerequisites

- docker ([installation instructions](#))
- nvidia-docker2 ([installation instructions](#))

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

9.4.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/yolov8
docker build --build-arg timezone='cat /etc/timezone' -t yolov8:v0 .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.
- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.

* This command will build the docker image with the necessary requirements using the Dockerfile exists in yolov8 directory.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all --ipc=host -v /path/to/
→local/data/dir:/path/to/docker/data/dir yolov8:v0
```

- `docker run` create a new docker container.

- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `yolov8:v0` the name of the docker image.

9.4.3. Training and exporting to ONNX

1. Train your model:

Once the docker is started, you can start training your model.

- Prepare your custom dataset - Follow the steps described [here](#) in order to create:
 - `dataset.yaml` configuration file
 - Labels - each image should have labels in YOLO format with corresponding txt file for each image.
 - Make sure to include number of classes field in the yaml, for example: `nc: 80`
- Start training - The following command is an example for training a *yolov8s* model.

```
yolo detect train data=coco128.yaml model=yolov8s.pt name=retrain_yolov8s
→ epochs=100 batch=16
```

- `yolov8s.pt` - pretrained weights. The pretrained weights for *yolov8n*, *yolov8s*, *yolov8m*, *yolov8l* and *yolov8x* will be downloaded to your working directory when running this command.
- `coco128.yaml` - example file for data.yaml file. Can be found at [ultralytics/ultralytics/datasets](#).
- `retrain_yolov8s` - the new weights will be saved at `ultralytics/runs/detect/retrain_yolov8s`.
- `epochs` - number of epochs to run. default to 100.
- `batch` - number of images per batch. default to 16.

NOTE: more configurable parameters can be found at <https://docs.ultralytics.com/modes/train/>

2. Export to ONNX:

In order to export your trained YOLOv8 model to ONNX run the following script:

```
yolo export model=/path/to/trained/best.pt imgsz=640 format=onnx opset=11 #
→ export at 640x640
```

NOTE: more configurable parameters can be found at <https://docs.ultralytics.com/modes/export/>

9.4.4. Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model.

In order to do so you need a working model-zoo environment.

Choose the corresponding YAML from our networks configuration directory, i.e.

`hailo_model_zoo/cfg/networks/yolov8s.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt yolov8s.onnx --calib-path /path/to/calibration/imgs/dir/ --
→yaml path/to/yolov8s.yaml --start-node-names name1 name2 --end-node-names name1 --
→classes 80
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- `--classes` - adjusting the number of classes in post-processing configuration (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note:

- Make sure to also update `preprocessing.input_shape` field on `yolo.yaml`, if it was changed on re-training.

More details about YAML files are presented [here](#).

9.5. YOLOX Retraining

- To learn more about yolox look [here](#)

9.5.1. Prerequisites

- docker ([installation instructions](#))
- nvidia-docker2 ([installation instructions](#))

NOTE: In case you use Hailo Software Suite docker, make sure you are doing all the following instructions outside of this docker.

9.5.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/yolox
docker build --build-arg timezone='cat /etc/timezone' -t yolox:v0 .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.
- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all -u "username" --ipc=host -v /
→path/to/local/data/dir:/path/to/docker/data/dir yolox:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `yolox:v0` the name of the docker image.

9.5.3. Training and exporting to ONNX

1. Prepare your data:

You can use coco format, which is already supported for training on your own custom dataset. More information can be found [here](#)

2. Training:

Start training with the following command:

```
python tools/train.py -f exps/default/yolox_s_leaky.py -d 8 -b 64 -c yolox_s.pth

exps/default/yolox_m_leaky.py
exps/default/yolox_l_leaky.py
exps/default/yolox_x_leaky.py
exps/default/yolox_s_wide_leaky.py
```

- `-f`: experiment description file
- `-d`: number of gpu devices
- `-b`: total batch size, the recommended number for `-b` is `num-gpu * 8`
- `-c`: path to pretrained weights which can be found in your working directory

```
_yolox_s.pth
_yolox_m.pth
_yolox_l.pth
_yolox_x.pth
```

3. Exporting to onnx:

After finishing training run the following command:

```
python tools/export_onnx.py --output-name yolox_s_leaky.onnx -f ./exps/default/
→yolox_s_leaky.py -c YOLOX_outputs/yolox_s_leaky/best_ckpt.pth
```

NOTE: Your trained model will be found under the following path: `/workspace/YOLOX/YOLOX_outputs/yolox_s_leaky/`, and the exported onnx will be written to `/workspace/YOLOX/yolox_s_leaky.onnx`

9.5.4. Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model. In order to do so you need a working model-zoo environment. Choose the corresponding YAML from our networks configuration directory, i.e. `hailo_model_zoo/cfg/networks/yolox_s_leaky.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt yolox_s_leaky.onnx --calib-path /path/to/calibration/imgs/
→dir/ --yaml path/to/yolox_s_leaky.yaml --start-node-names name1 name2 --end-node-
→names name1 --classes 80
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- `--classes` - adjusting the number of classes in post-processing configuration (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note: More details about YAML files are presented [here](#).

9.6. DAMO-YOLO Retraining

- To learn more about DAMO-YOLO visit the [official repository](#)

9.6.1. Prerequisites

- docker ([docker installation instructions](#))
- nvidia-docker2 ([nvidia docker installation instructions](#))

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

9.6.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/damoyolo
docker build --build-arg timezone='cat /etc/timezone' -t damoyolo:v0 .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.
- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.

* This command will build the docker image with the necessary requirements using the Dockerfile exists in damoyolo directory.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all --ipc=host -v /path/to/
→local/data/dir:/path/to/docker/data/dir damoyolo:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `damoyolo:v0` the name of the docker image.

9.6.3. Training and exporting to ONNX

1. Train your model:

Once the docker is started, you can start training your model.

- Prepare your custom dataset (must be coco format) - Follow the steps described [here](#).
- Modify `num_classes` and `class_names` in the configuration file, for example `damoyolo_tinynasL20_T.py`
- Use `self.train.batch_size / self.train.total_epochs` in the configuration file to modify the `batch_size` and number of epochs
- Update the symbolic link to your dataset: `ln -sf /your/coco/like/dataset/path datasets/coco`
- Start training - The following command is an example for training a `damoyolo_tinynasL20_T` model.

```
python tools/train.py -f configs/damoyolo_tinynasL20_T.py
```

```
configs/damoyolo_tinynasL25_S.py
configs/damoyolo_tinynasL35_M.py
```

- `configs/damoyolo_tinynasL20_T.py` - configuration file of the DAMO-YOLO variant you would like to train. In order to change the number of classes make sure you update `num_classes` and `class_names` in this file.

2. Export to ONNX:

In order to export your trained DAMO-YOLO model to ONNX run the following script:

```
python tools/converter.py -f configs/damoyolo_tinynasL20_T.py -c /path/to/
→trained/model.pth --batch_size 1 --img_size 640 # export at 640x640 with batch
→size 1
```

9.6.4. Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model.

In order to do so you need a working model-zoo environment.

Choose the corresponding YAML from our networks configuration directory, i.e.

`hailo_model_zoo/cfg/networks/damoyolo_tinynasL20_T.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt damoyolo_tinynasL20_T.onnx --calib-path /path/to/
→calibration/imgs/dir/ --yaml path/to/damoyolo/variant.yaml --start-node-names
→name1 name2 --end-node-names name1
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

More details about YAML files are presented [here](#).

9.7. Nanodet Retraining

- To learn more about NanoDet look [here](#)

9.7.1. Prerequisites

- docker ([installation instructions](#))
- nvidia-docker2 ([installation instructions](#))

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

9.7.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/nanodet
docker build -t nanodet:v0 --build-arg timezone=`cat /etc/timezone` .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.
- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all -u "username" --ipc=host -v /
→path/to/local/data/dir:/path/to/docker/data/dir nanodet:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-u <username>` same username as used for building the image.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `nanodet:v0` the name of the docker image.

9.7.3. Training and exporting to ONNX

1. Prepare your data:

Data is expected to be in coco format. More information can be found [here](#)

2. Training:

Configure your model in a .yaml file. We'll use

/workspace/nanodet/config/legacy_v0.x_configs/RepVGG/nanodet-RepVGG-A0_416.yaml in this guide.

Modify the path for the dataset in the .yaml configuration file:

```
data:
  train:
    name: CocoDataset
    img_path: <path-to-train-dir>
    ann_path: <path-to-annotations-file>
    ...
  val:
    name: CocoDataset
    img_path: <path-to-validation-dir>
    ann_path: <path-to-annotations-file>
    ...
```

Start training with the following commands:

```
cd /workspace/nanodet
ln -s /workspace/data/coco/ /coco
python tools/train.py ./config/legacy_v0.x_configs/RepVGG/nanodet-RepVGG-A0_
↪416.yaml
```

In case you want to use the pretrained nanodet-RepVGG-A0_416.ckpt, which was predownloaded into your docker modify your configurationf file:

```
schedule:
  load_model: ./pretrained/nanodet-RepVGG-A0_416.ckpt
```

Modifying the batch size and the number of GPUs used for training can be done also in the configuration file:

```
device:
  gpu_ids: [0]
  workers_per_gpu: 1
  batchsize_per_gpu: 128
```

3. Exporting to onnx

After training, install the ONNX and ONNXruntime packages, then export the ONNX model:

```
python tools/export_onnx.py --cfg_path ./config/legacy_v0.x_configs/RepVGG/  
→nanodet-RepVGG-A0_416.yml --model_path /workspace/nanodet/workspace/RepVGG-  
→A0-416/model_last.ckpt
```

NOTE: Your trained model will be found under the following path: /workspace/nanodet/workspace/<backbone-name>/model_last.ckpt, and exported onnx will be written to /workspace/nanodet/nanodet.onnx

9.7.4. Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model.

In order to do so you need a working model-zoo environment.

Choose the corresponding YAML from our networks configuration directory, i.e.

hailo_model_zoo/cfg/networks/nanodet_repvgg.yml, and run compilation using the model zoo:

```
hailomz compile --ckpt nanodet.onnx --calib-path /path/to/calibration/imgs/dir/ --  
→yaml path/to/nanodet_repvgg.yml --start-node-names name1 name2 --end-node-names  
→name1 --classes 80
```

- --ckpt - path to your ONNX file.
- --calib-path - path to a directory with your calibration images in JPEG/png format
- --yaml - path to your configuration YAML file.
- --start-node-names and --end-node-names - node names for customizing parsing behavior (optional).
- --classes - adjusting the number of classes in post-processing configuration (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note:

- On your desired YAML file, change preprocessing.input_shape if changed on retraining.

More details about YAML files are presented [here](#).

9.8. FCN Retraining

- To learn more about FCN look [here](#)
-

9.8.1. Prerequisites

- [docker \(installation instructions\)](#)
- [nvidia-docker2 \(installation instructions\)](#)

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

9.8.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/fcn
docker build -t fcn:v0 --build-arg timezone='cat /etc/timezone' .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.
- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all -u "username" --ipc=host -v /
→path/to/local/data/dir:/path/to/docker/data/dir fcn:v0
```

- `docker run` create a new docker container.
- `--name <docker-name>` name for your container.
- `-u <username>` same username as used for building the image.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `fcn:v0` the name of the docker image.

9.8.3. Training and exporting to ONNX

1. Prepare your data:

Data is expected to be in coco format, and by default should be in `/workspace/data/<dataset_name>`.
The expected structure is as follows:

```

/workspace
|-- mmsegmentation
`-- |-- data
    |-- cityscapes
        |-- gtFine
            |-- train
            |-- | -- aachem
            |-- | -- | -- *.png
            |-- | `-- ...
            |-- `-- test
            |-- | -- berlin
            |-- | -- | -- *.png
            |-- | `-- ...
        |-- leftImg8bit
            |-- train
            |-- | -- aachem
            |-- | -- | -- *.png
            |-- | `-- ...
        |-- test
            |-- berlin
            |-- | -- *.png
            |-- `-- ...

```

more information can be found [here](#)

2. Training:

Configure your model in a .py file. We'll use /workspace/mmsegmentation/configs/fcn/fcn8_r18_hailo.py in this guide.

start training with the following command:

```

cd /workspace/mmsegmentation
./tools/dist_train.sh configs/fcn/fcn8_r18_hailo.py 2

```

Where 2 is the number of GPUs used for training.

3. Exporting to onnx

After training, run the following command:

```

cd /workspace/mmsegmentation
python ./tools/pytorch2onnx.py configs/fcn/fcn8_r18_hailo.py --checkpoint ./
➔work_dirs/fcn8_r18_hailo/iter_59520.pth --shape 1024 1920 --out_name fcn.onnx

```

9.8.4. Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model.

In order to do so you need a working model-zoo environment.

Choose the corresponding YAML from our networks configuration directory, i.e.

`hailo_model_zoo/cfg/networks/fcn8_resnet_v1_18.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt fcn.onnx --calib-path /path/to/calibration/imgs/dir/ --yaml
↳ path/to/fcn8_resnet_v1_18.yaml --start-node-names name1 name2 --end-node-names
↳ name1
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note: More details about YAML files are presented [here](#).

9.9. YOLACT Retraining

- To learn more about Yolact look [here](#)

9.9.1. Prerequisites

- docker ([installation instructions](#))
- nvidia-docker2 ([installation instructions](#))

NOTE: In case you use Hailo Software Suite docker, make sure you are doing all the following instructions outside of this docker.

9.9.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/yolact
docker build --build-arg timezone='cat /etc/timezone' -t yolact:v0 .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.

- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.
- This command will build the docker image with the necessary requirements using the Dockerfile exists in the yolact directory.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all --ipc=host -v /path/to/local/  
data/dir:/path/to/docker/data/dir yolact:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `yolact:v0` the name of the docker image.

9.9.3. Training and exporting to ONNX

1. Prepare your data:

Data is expected to be in coco format, and by default should be in `/workspace/data/<dataset_name>`.

The expected structure is as follows:

```
/workspace  
|-- data  
`-- |-- coco  
    |-- annotations  
    | |-- instances_train2017.json  
    | |-- instances_val2017.json  
    | |-- person_keypoints_train2017.json  
    | |-- person_keypoints_val2017.json  
    | |-- image_info_test-dev2017.json  
    |-- images  
    | |-- train2017  
    | | |-- *.jpg  
    | |-- val2017  
    | | |-- *.jpg  
    |-- test2017  
    | |-- *.jpg
```

For training on custom datasets see [here](#)

2. Train your model:

Once your dataset is prepared, create a soft link to it under the yolact/data work directory, then you can start training your model:

```
cd /workspace/yolact
ln -s /workspace/data/coco data/coco
python train.py --config=yolact_regnetx_800MF_config
```

- `yolact_regnetx_800MF_config` - configuration using the `regnetx_800MF` backbone.

3. Export to ONNX: In order to export your trained YOLACT model to ONNX run the following script:

```
python export.py --config=yolact_regnetx_800MF_config --trained_model=path/to/
→trained/model --export_path=path/to/export/model.onnx
```

- `--config` - same configuration used for training.
- `--trained_model` - path to the weights produced by the training process.
- `--export_path` - path to export the ONNX file to. Include the `.onnx` extension.

9.9.4. Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model. In order to do so you need a working model-zoo environment. Choose the corresponding YAML from our networks configuration directory, i.e. `hailo_model_zoo/cfg/networks/yolact.yaml`, and run compilation using the model zoo:

```
hailomz compile yolact --ckpt yolact.onnx --calib-path /path/to/calibration/imgs/
→dir/ --yaml path/to/yolact_regnetx_800mf_20classes.yaml --start-node-names name1
→name2 --end-node-names name1
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note:

- The `'yolact_regnetx_800mf_20classes.yaml'` https://github.com/hailo-ai/hailo_model_zoo/blob/master/hailo_model_zoo/cfg/yolact_regnetx_800mf_20classes.yaml is an example yaml where some of the classes (out of 80) were removed. If you wish to change the number of classes, the easiest way is to retrain with the exact number of classes, erase the `channels_remove` section (lines 18 to 437).

More details about YAML files are presented [here](#).

9.10. YOLOv8-seg Retraining

- To learn more about yolov8 look [here](#)

9.10.1. Prerequisites

- [docker](#) ([installation instructions](#))
- [nvidia-docker2](#) ([installation instructions](#))

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

9.10.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/yolov8_seg
docker build --build-arg timezone='cat /etc/timezone' -t yolov8_seg:v0 .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.
- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.

* This command will build the docker image with the necessary requirements using the Dockerfile exists in yolov8-seg directory.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all --ipc=host -v /path/to/
→local/data/dir:/path/to/docker/data/dir yolov8_seg:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `yolov8:v0` the name of the docker image.

9.10.3. Training and exporting to ONNX

1. Train your model:

Once the docker is started, you can start training your model.

- Prepare your custom dataset - Follow the steps described [here](#) in order to create:
 - `dataset.yaml` configuration file
 - Labels - each image should have labels in YOLO format with corresponding txt file for each image.
 - Make sure to include number of classes field in the yaml, for example: `nc: 80`
- Start training - The following command is an example for training a `yolov8s-seg` model.

```
yolo segment train data=coco128-seg.yaml model=yolov8s-seg.pt name=retrain_
→yolov8s_seg epochs=100 batch=16
```

- `yolov8s-seg.pt` - pretrained weights. The pretrained weights for `yolov8n-seg`, `yolov8s-seg`, `yolov8m-seg`, `yolov8l-seg` and `yolov8x-seg` will be downloaded to your working directory when running this command.
- `coco128-seg.yaml` - example file for `data.yaml` file. Can be found at ultralytics.com/datasets.
- `retrain_yolov8s_seg` - the new weights will be saved at `ultralytics/runs/segment/retrain_yolov8s_seg`.
- `epochs` - number of epochs to run. default to 100.
- `batch` - number of images per batch. default to 16.

NOTE: more configurable parameters can be found at <https://docs.ultralytics.com/modes/train/>

2. Export to ONNX:

In order to export your trained YOLOv8-seg model to ONNX run the following script:

```
yolo export model=/path/to/trained/best.pt imgsz=640 format=onnx opset=11 #
→export at 640x640
```

NOTE: more configurable parameters can be found at <https://docs.ultralytics.com/modes/export/>

9.10.4. Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model.

In order to do so you need a working model-zoo environment.

Choose the corresponding YAML from our networks configuration directory, i.e.

`hailo_model_zoo/cfg/networks/yolov8s-seg.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt yolov8s-seg.onnx --calib-path /path/to/calibration/imgs/dir/
→ --yaml path/to/yolov8s-seg.yaml --start-node-names name1 name2 --end-node-names
→name1
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.

- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note:

- Make sure to also update `preprocessing.input_shape` field on `yolo.yaml`, if it was changed on re-training.

More details about YAML files are presented [here](#).

9.11. Centerpose Retraining

- To learn more about CenterPose look [here](#)

9.11.1. Prerequisites

- docker ([installation instructions](#))
- nvidia-docker2 ([installation instructions](#))

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

9.11.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/centerpose
docker build -t centerpose:v0 --build-arg timezone='cat /etc/timezone' .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.
- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all -u "username" --ipc=host -v /
↳path/to/local/data/dir:/path/to/docker/data/dir centerpose:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.

- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `centerpose:v0` the name of the docker image.

9.11.3. Training and exporting to ONNX

1. Prepare your data:

Data is expected to be in coco format, and by default should be in `/workspace/data/<dataset_name>`.

The expected structure is as follows:

```
/workspace
|-- data
`-- |-- coco
    |-- |-- annotations
        |-- |-- instances_train2017.json
        |-- |-- instances_val2017.json
        |-- |-- person_keypoints_train2017.json
        |-- |-- person_keypoints_val2017.json
        |-- |-- image_info_test-dev2017.json
    |-- |-- images
        |-- |-- train2017
        |-- |-- |-- *.jpg
        |-- |-- val2017
        |-- |-- |-- *.jpg
        |-- |-- test2017
        |-- |-- |-- *.jpg
```

The path for the dataset can be configured in the .yaml file, e.g. `centerpose/experiments/regnet_fpn.yaml`

2. Training:

Configure your model in a .yaml file. We'll use `/workspace/centerpose/experiments/regnet_fpn.yaml` in this guide.

start training with the following command:

```
cd /workspace/centerpose/tools
python -m torch.distributed.launch --nproc_per_node 4 train.py --cfg ../
→experiments/regnet_fpn.yaml
```

Where 4 is the number of GPUs used for training.

If using a different number, update both this and the used gpus in the .yaml configuration.

3. Exporting to onnx After training, run the following command:

```
cd /workspace/centerpose/tools
python export.py --cfg ../experiments/regnet_fpn.yaml --TESTMODEL /workspace/
→out/regnet1_6/model_best.pth
```

9.11.4. Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model. In order to do so you need a working model-zoo environment. Choose the corresponding YAML from our networks configuration directory, i.e. `hailo_model_zoo/cfg/networks/centerpose_regnetx_1.6gf_fpn.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt coco_pose_regnet1.6_fpn.onnx --calib-path /path/to/
→calibration/imgs/dir/ --yaml path/to/centerpose_regnetx_1.6gf_fpn.yaml --start-
→node-names name1 name2 --end-node-names name1
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note: More details about YAML files are presented [here](#).

9.12. MSPN Retraining

9.12.1. Prerequisites

- `docker` ([installation instructions](#))
- `nvidia-docker2` ([installation instructions](#))

NOTE: In case you are using the Hailo Software Suite docker, make sure to run all of the following instructions outside of that docker.

9.12.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/mspn
docker build -t mspn:v0 --build-arg timezone=`cat /etc/timezone` .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.

- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all -u "username" --ipc=host -v /
→path/to/local/data/dir:/path/to/docker/data/dir mspn:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `centerpose:v0` the name of the docker image.

9.12.3. Training and exporting to ONNX

1. Prepare your data:

Data is expected to be in coco format, and by default should be in `/workspace/data/<dataset_name>`.
The expected structure is as follows:

```
/workspace
|-- data
`-- |-- coco
    |-- |-- annotations
        |-- |-- instances_train2017.json
        |-- |-- instances_val2017.json
        |-- |-- person_keypoints_train2017.json
        |-- |-- person_keypoints_val2017.json
        |-- |-- image_info_test-dev2017.json

    `---|-- train2017
        |---|---|-- *.jpg
    `---|-- val2017
        |---|---|-- *.jpg
    `---|-- test2017
        |---|---|-- *.jpg
```

The path for the dataset can be configured in the `.py` config file, e.g. `configs/body/2d_kpt_sview_rgb_img/topdown_heatmap/coco/regnetx_800mf_256x192.py`

2. Training:

Configure your model in a `.py` config file. We will use `/workspace/mmpose/configs/body/2d_kpt_sview_rgb_img/topdown_heatmap/coco/regnetx_800mf_256x192.py` in this guide. Start training with the following command:

```
cd /workspace/mmpose
./tools/dist_train.sh ./configs/body/2d_kpt_sview_rgb_img/topdown_heatmap/
→coco/regnetx_800mf_256x192.py 4 --work-dir exp0
```


Where 4 is the number of GPUs used for training. In this example, the trained model will be saved under `exp0` directory.

3. Export to onnx

In order to export your trained model to ONNX run the following script:

```
cd /workspace/mmpose
python tools/deployment/pytorch2onnx.py ./configs/body/2d_kpt_sview_rgb_img/
↳topdown_heatmap/coco/regnetx_800mf_256x192.py exp0/best_AP_epoch_310.pth --
↳output-file mspn_regnetx_800mf.onnx
```

where `exp0/best_AP_epoch_310.pth` should be replaced by the trained model file path.

9.12.4. Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model.

In order to do so you need a working model-zoo environment.

Choose the corresponding YAML from our networks configuration directory, i.e.

`hailo_model_zoo/cfg/networks/mspn_regnetx_800mf.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt mspn_regnetx_800mf.onnx --calib-path /path/to/calibration/
↳imgs/dir/ --yaml path/to/mspn_regnetx_800mf.yaml --start-node-names name1 name2 --
↳end-node-names name1
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).
- The model zoo will take care of adding the input normalization to be part of the model.

Note: More details about YAML files are presented [here](#).

9.13. Arcface Retraining

- To learn more about arcface look [here](#)

9.13.1. Prerequisites

- docker ([installation instructions](#))
- nvidia-docker2 ([installation instructions](#))

NOTE: In case you use Hailo Software Suite docker, make sure you are doing all the following instructions outside of this docker.

9.13.2. Environment Preparations

1. Build the docker image:

```
cd hailo_model_zoo/training/arcface
docker build --build-arg timezone='cat /etc/timezone' -t arcface:v0 .
```

the following optional arguments can be passed via `--build-arg`:

- `timezone` - a string for setting up timezone. E.g. "Asia/Jerusalem"
- `user` - username for a local non-root user. Defaults to 'hailo'.
- `group` - default group for a local non-root user. Defaults to 'hailo'.
- `uid` - user id for a local non-root user.
- `gid` - group id for a local non-root user.

2. Start your docker:

```
docker run --name "your_docker_name" -it --gpus all -u "username" --ipc=host -v /
→path/to/local/data/dir:/path/to/docker/data/dir arcface:v0
```

- `docker run` create a new docker container.
- `--name <your_docker_name>` name for your container.
- `-it` runs the command interactively.
- `--gpus all` allows access to all GPUs.
- `--ipc=host` sets the IPC mode for the container.
- `-v /path/to/local/data/dir:/path/to/docker/data/dir` maps `/path/to/local/data/dir` from the host to the container. You can use this command multiple times to mount multiple directories.
- `arcface:v0` the name of the docker image.

9.13.3. Training and exporting to ONNX

1. Prepare your data:

For more information on obtaining datasets [here](#)

The repository supports the following formats:

1. ImageFolder dataset - each class (person) has its own directory
Validation data is packed .bin files

```
data_dir/
  agedb_30.bin
  cfp_fp.bin
  lfw.bin
  person0/
  person1/
  ...
  personlast/
```

2. MxNetRecord - train.rec and train.idx files. This is the format of insightface datasets. Validation data is packed .bin files

```
data_dir/
  agedb_30.bin
  cfp_fp.bin
  lfw.bin
  train.idx
  train.rec
```

2. Training:

Start training with the following command:

```
python -m torch.distributed.launch --nproc_per_node=2 --nnodes=1 --node_rank=0
--master_addr="127.0.0.1" --master_port=12581 train_v2.py /path/to/config
```

- nproc_per_node: number of gpu devices

3. Exporting to onnx:

After finishing training run the following command:

```
python torch2onnx.py /path/to/model.pt --network mbf --output /path/to/model.
--onnx --simplify true
```

9.13.4. Compile the Model using Hailo Model Zoo

You can generate an HEF file for inference on Hailo-8 from your trained ONNX model. In order to do so you need a working model-zoo environment. Choose the corresponding YAML from our networks configuration directory, i.e. `hailo_model_zoo/cfg/networks/arcface_mobilefacenet.yaml`, and run compilation using the model zoo:

```
hailomz compile --ckpt arcface_s_leaky.onnx --calib-path /path/to/calibration/imgs/
--dir/ --yaml /path/to/arcface_mobilefacenet.yaml --start-node-names name1 name2 --
--end-node-names name1
```

- `--ckpt` - path to your ONNX file.
- `--calib-path` - path to a directory with your calibration images in JPEG/png format
- `--yaml` - path to your configuration YAML file.
- `--start-node-names` and `--end-node-names` - node names for customizing parsing behavior (optional).

- The model zoo will take care of adding the input normalization to be part of the model.

Note: More details about YAML files are presented [here](#).
